



Lecture 15: Shape Deformation

Li Yi

June 5, 2025

Recap

- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction

Intrinsic-Extrinsic Ratio for Triangles

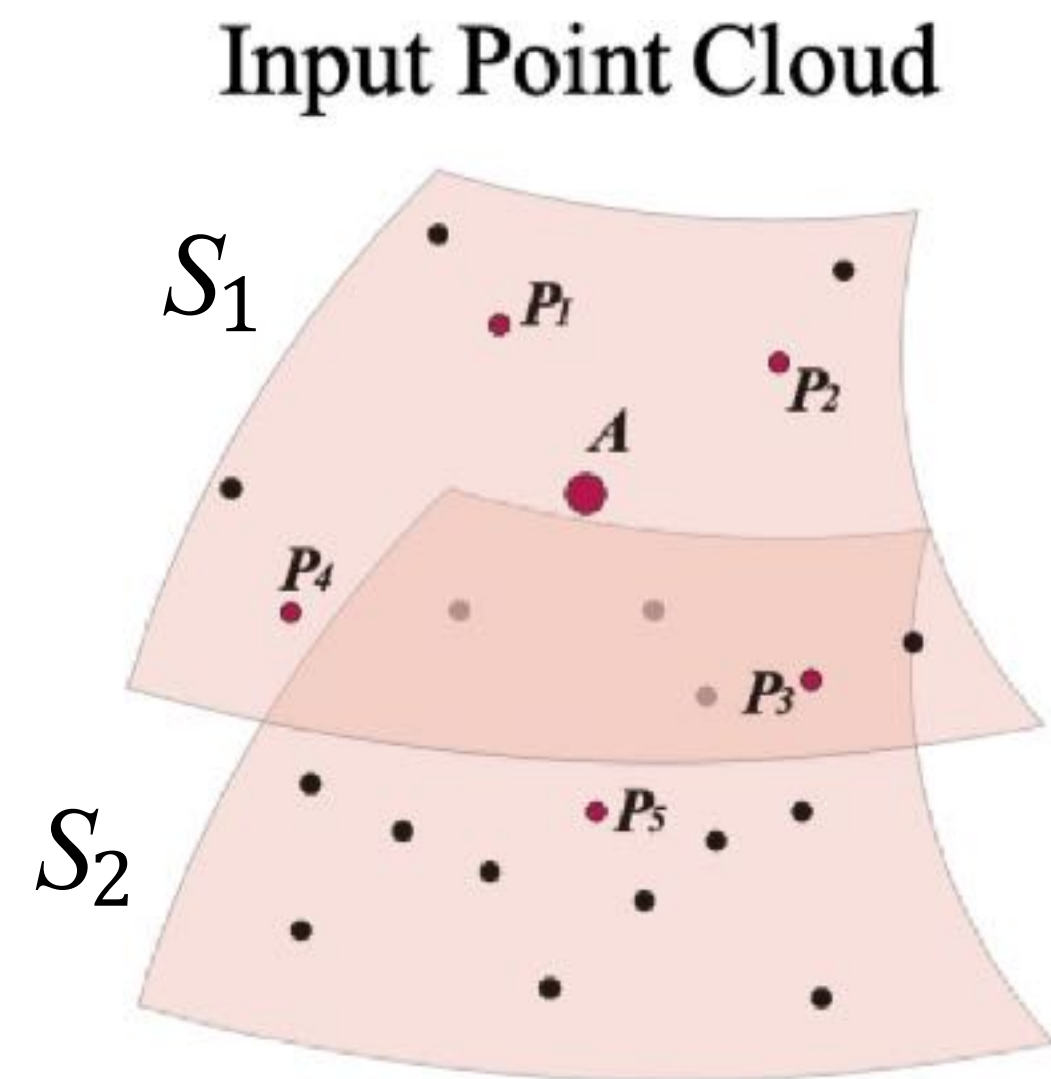
For a triangle $\triangle uvw$:

$$\text{IER}(\triangle uvw) = \frac{d_G(u, v) + d_G(v, w) + d_G(w, u)}{d_E(u, v) + d_E(v, w) + d_E(w, u)}$$

If point u, v, w is **close enough**, $\text{IER}(\triangle uvw) \leq 1 + \varepsilon$
 $\Rightarrow$ (very likely) Triangle is on the surface

We learn to predict this
quantity!

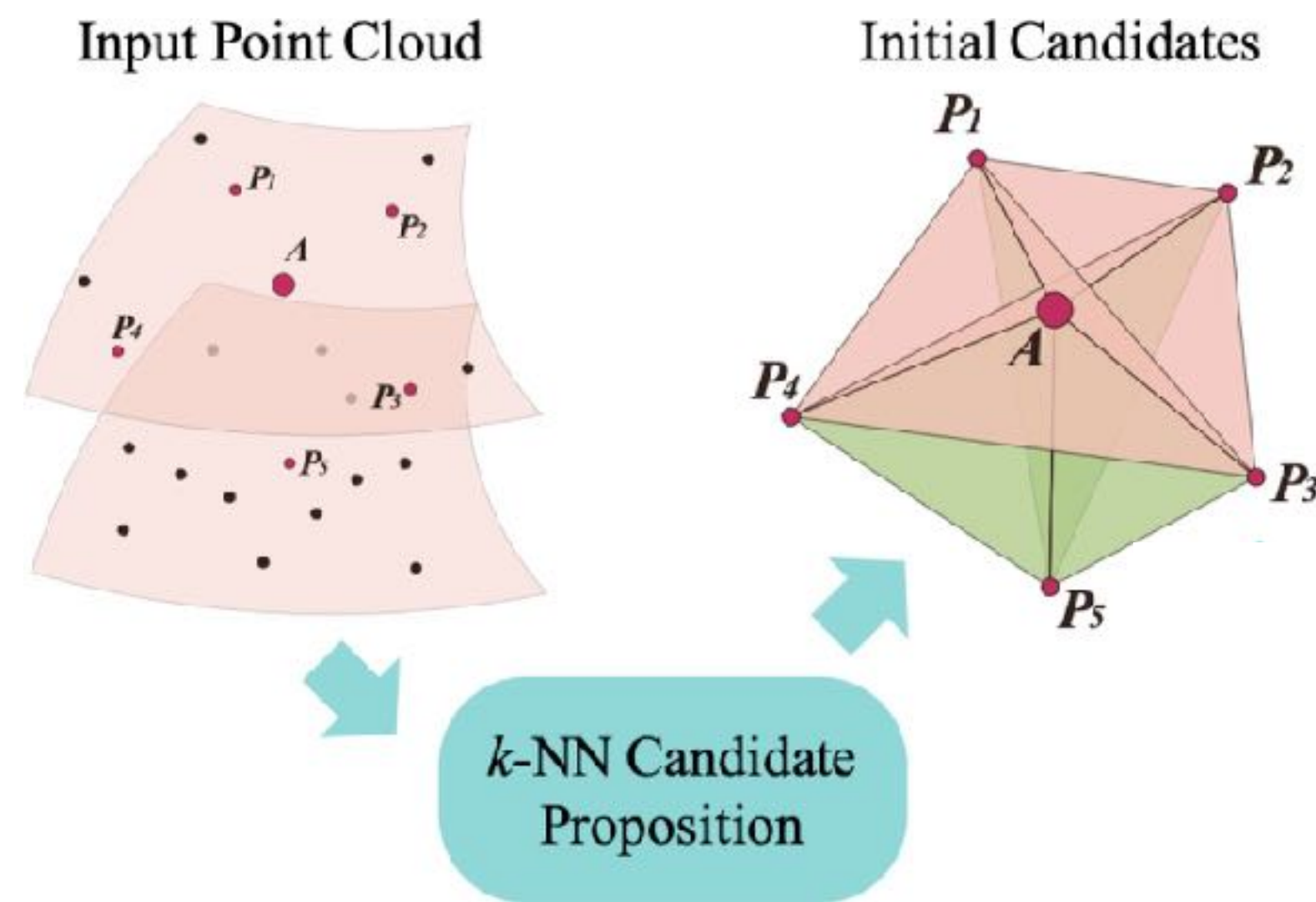
Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



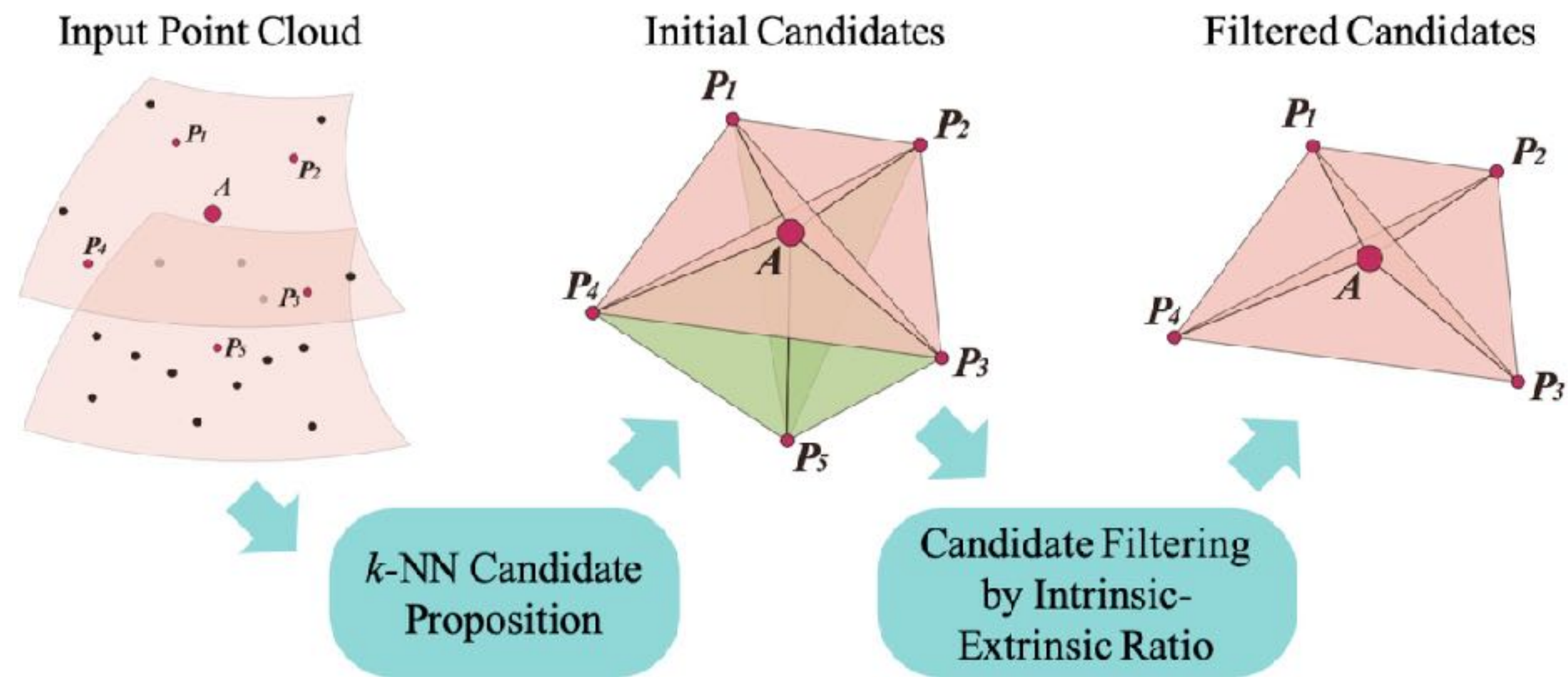
Case Study:

- Input point cloud is sampled from two closed thin surfaces S_1 and S_2
- $\{P_i\}$ are Euclidean nearest neighbors of
 - $\{A, P_1, P_2, P_3, P_4\} \subset S_1$
 - $P_5 \in S_1$

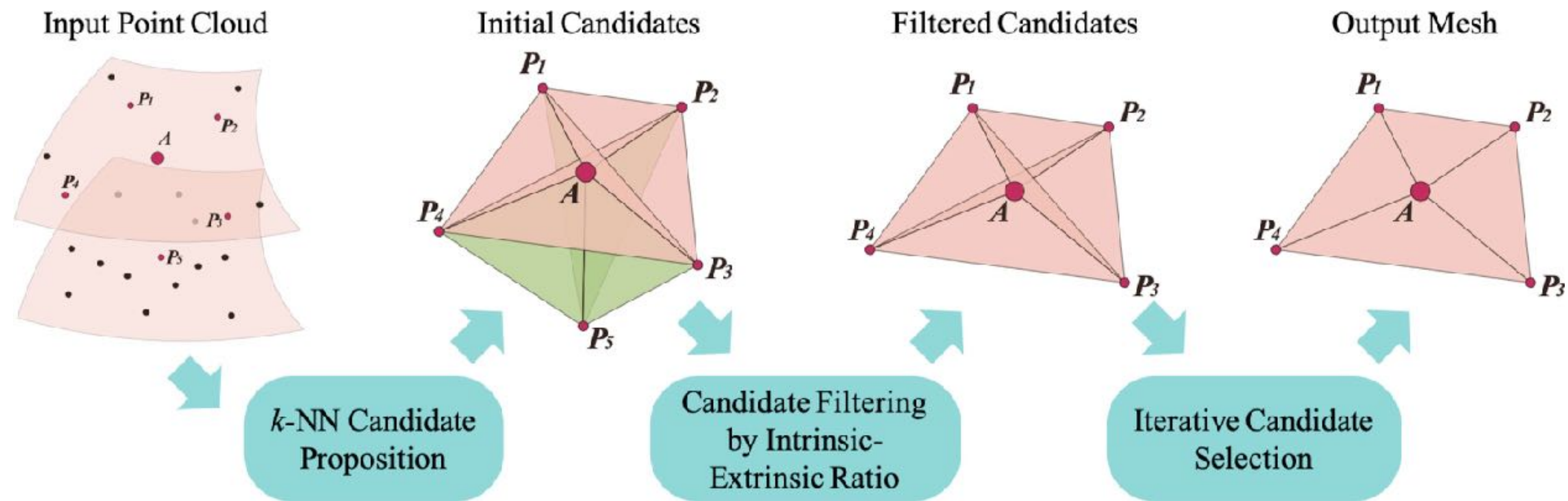
Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance

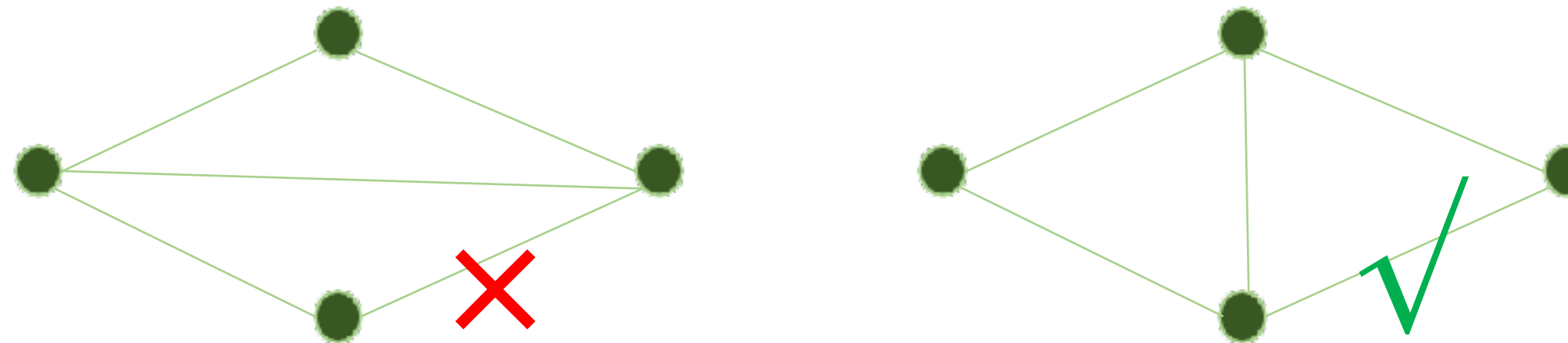


Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



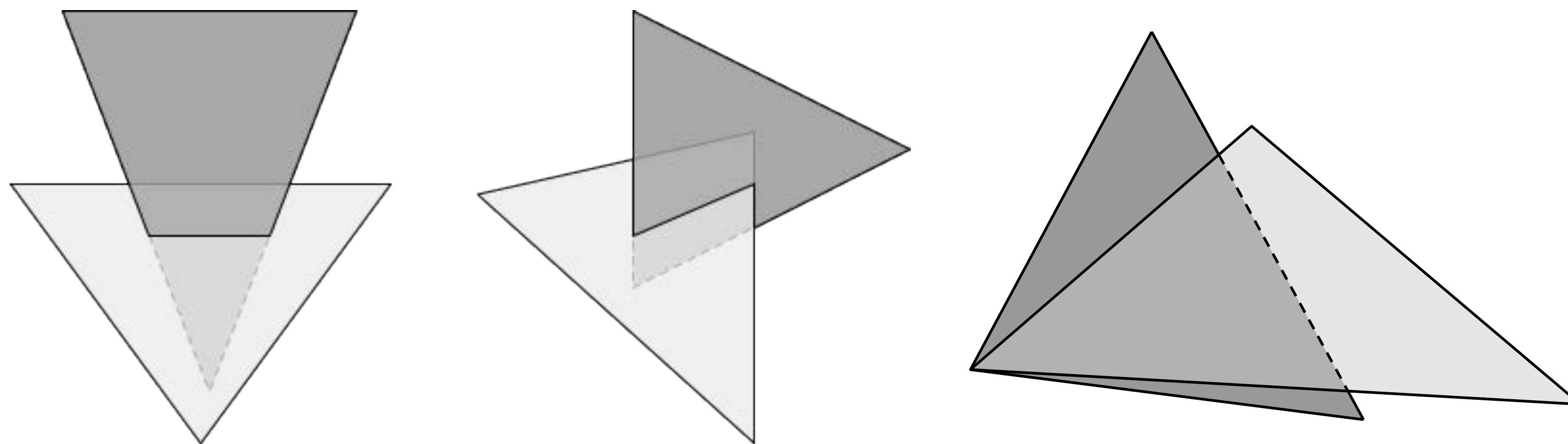
Iterative Candidate Selection

- The added triangle should not break manifold properties
 - No edge has more than two incident faces
 - No intersection between triangles
- Prefer equilateral triangles:



Measured by the ratio of longest edge to shortest edge

Triangle Collision Detection



Guigue P, Devillers O. Fast and robust triangle-triangle overlap test using orientation predicates. Journal of Graphics Tools. 2003 Jan 1;8(1):25-32.

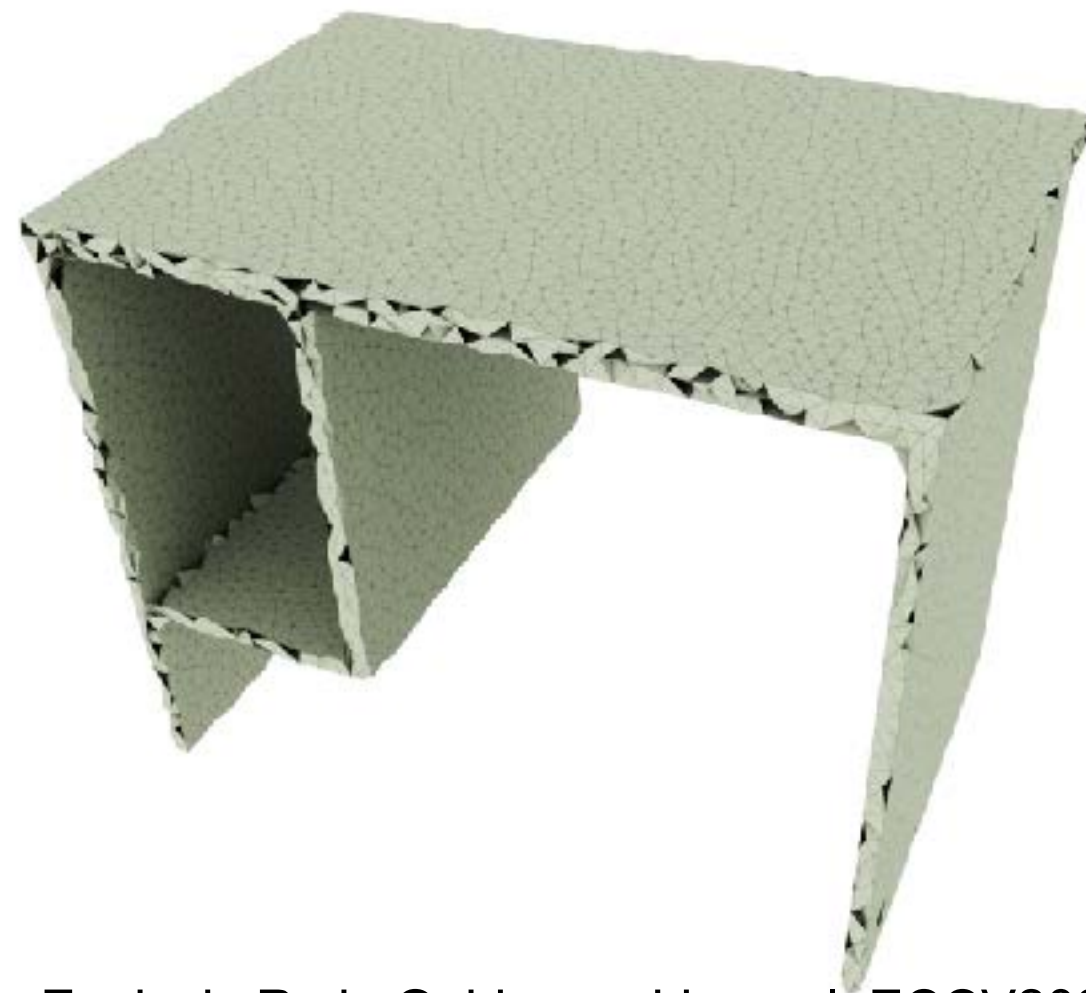
Code: https://github.com/erich666/jgt-code/tree/master/Volume_08/Number_1/Guigue2003

Meshing Results

k -NN

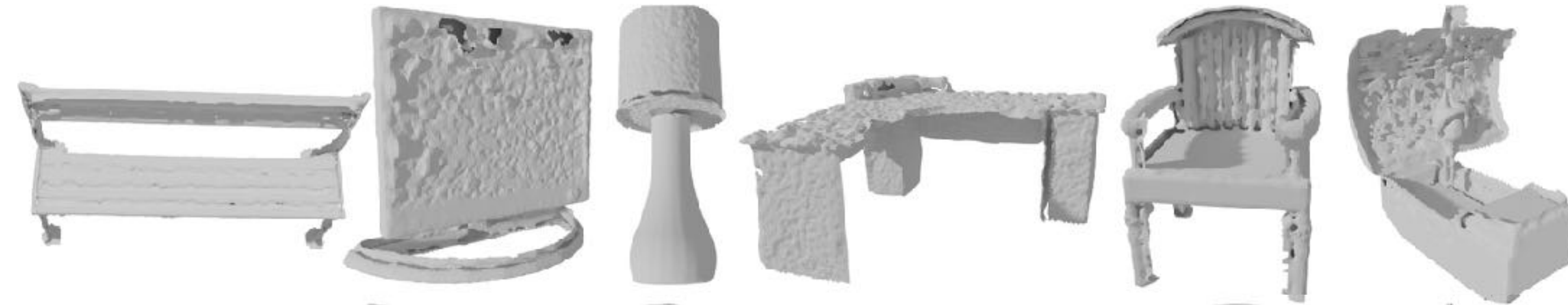


k -NN
with IER

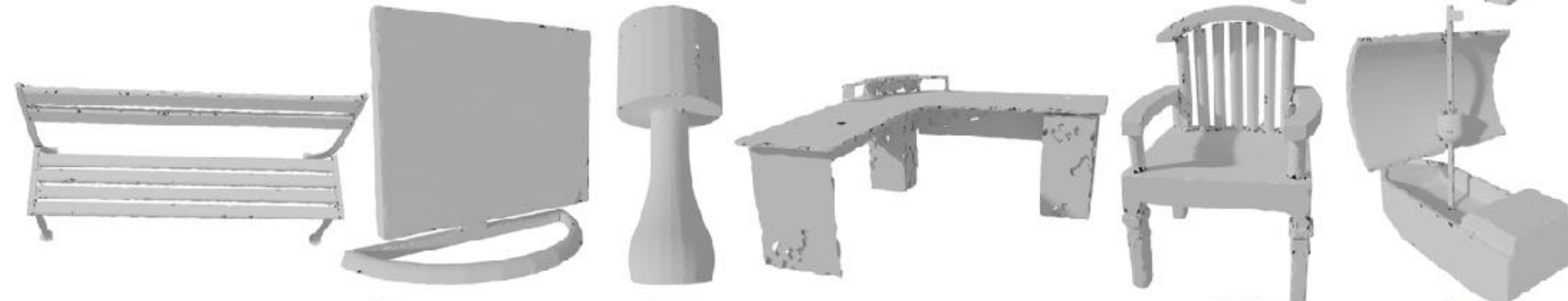


Robustness

Poisson



k -NN
with IER

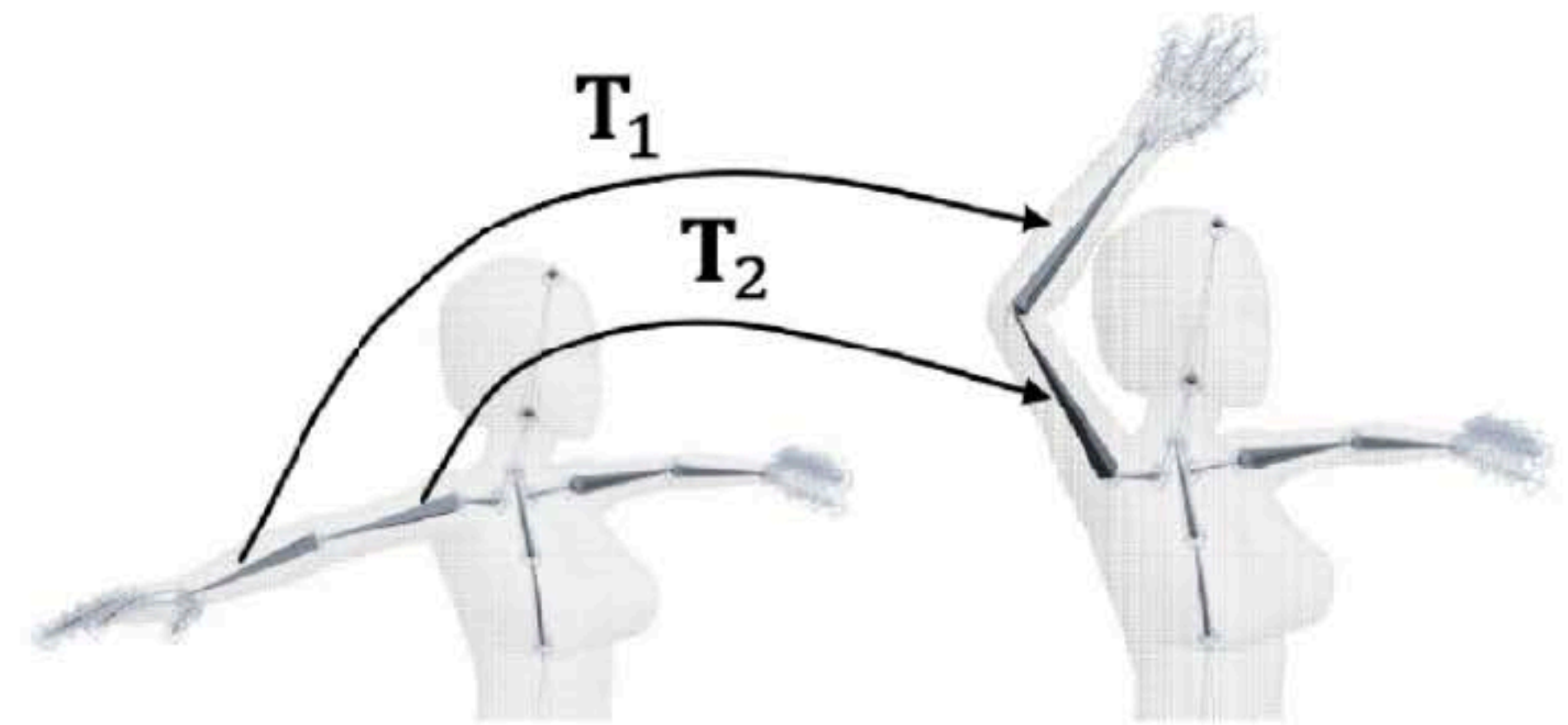
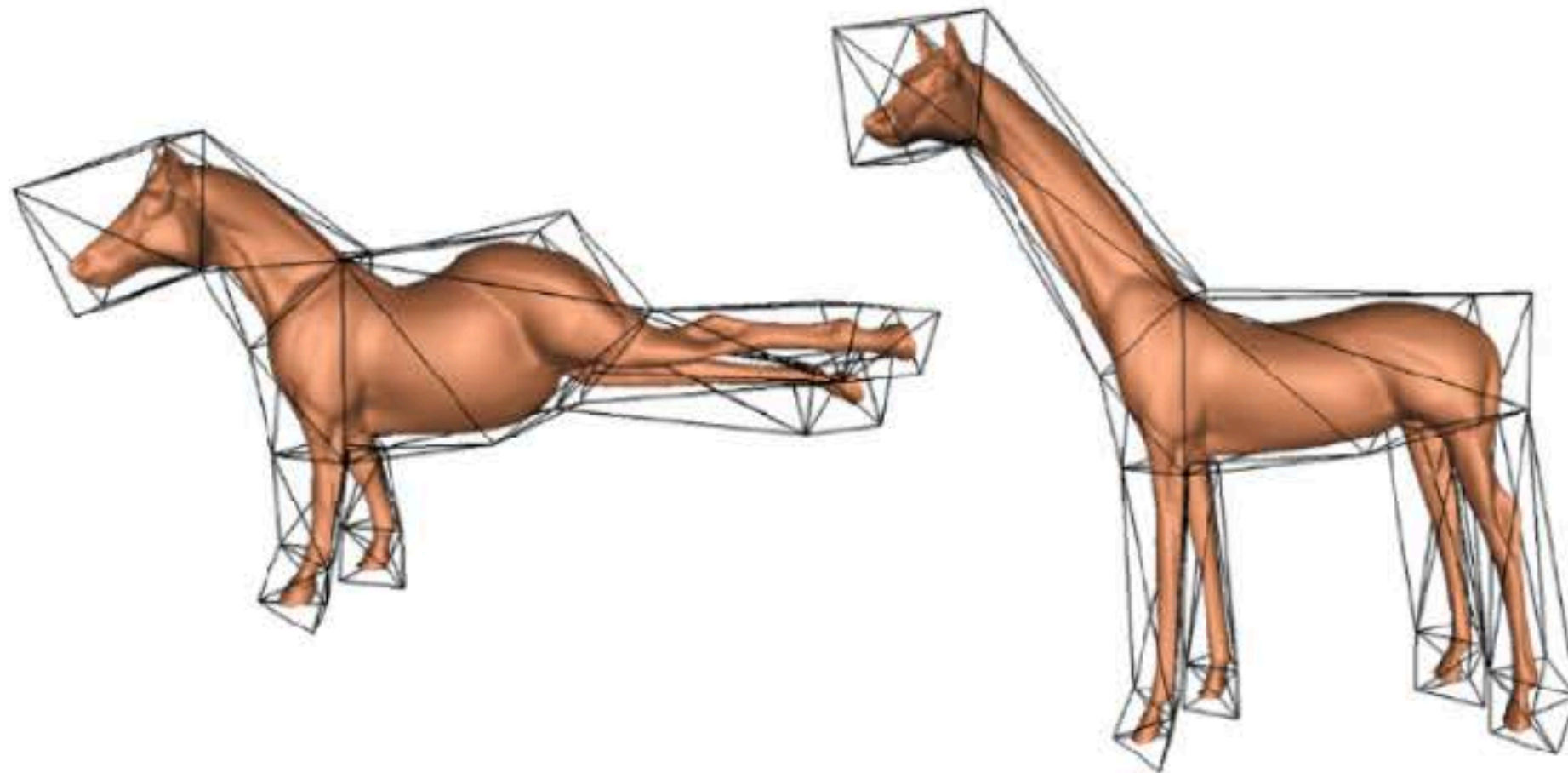
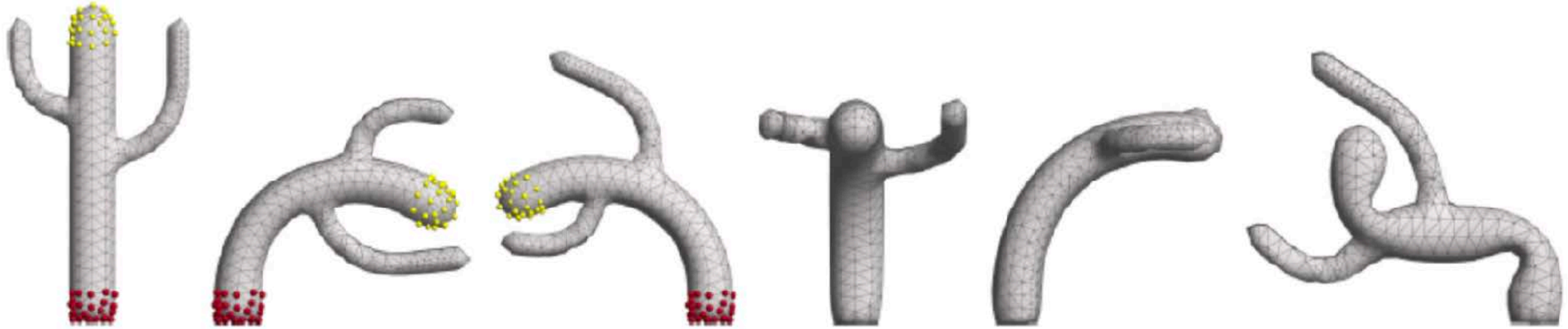


Ground-
Truth



Today's Focus

- 3D analysis - shape deformation



Outline

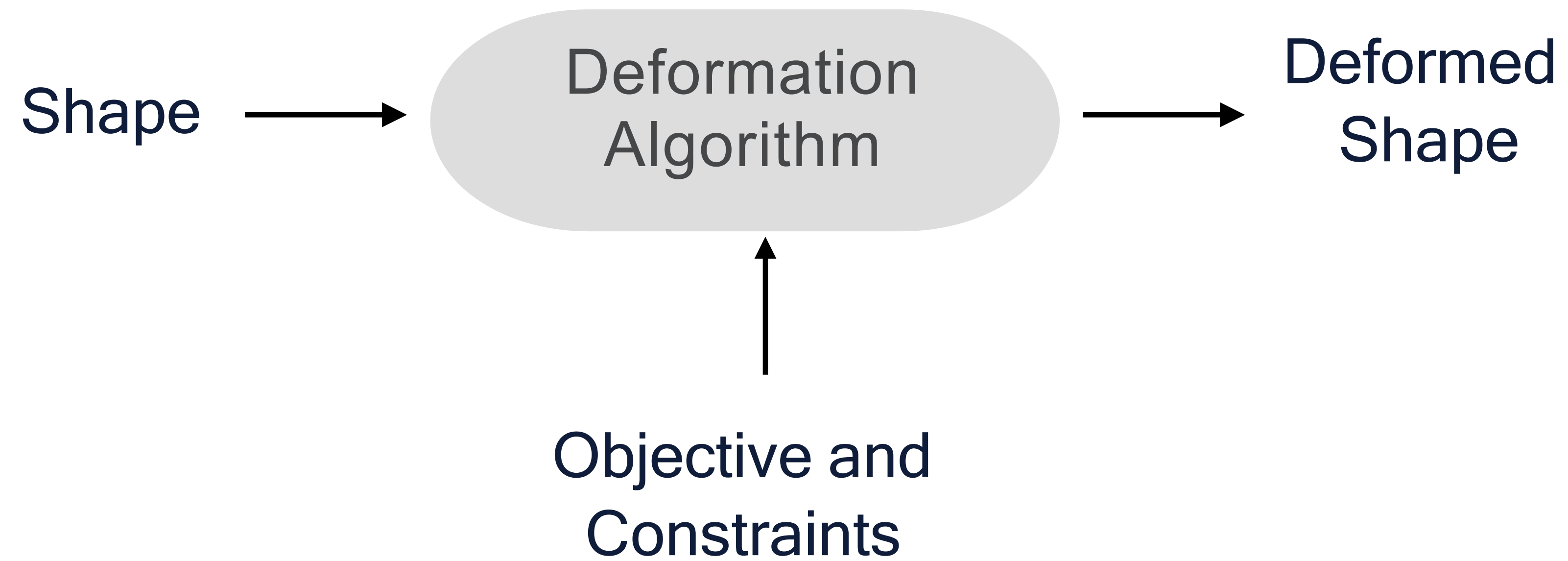
- Introduction
- Surface Deformation
- Space Deformation
- Skeleton Skinning

Shape Deformation

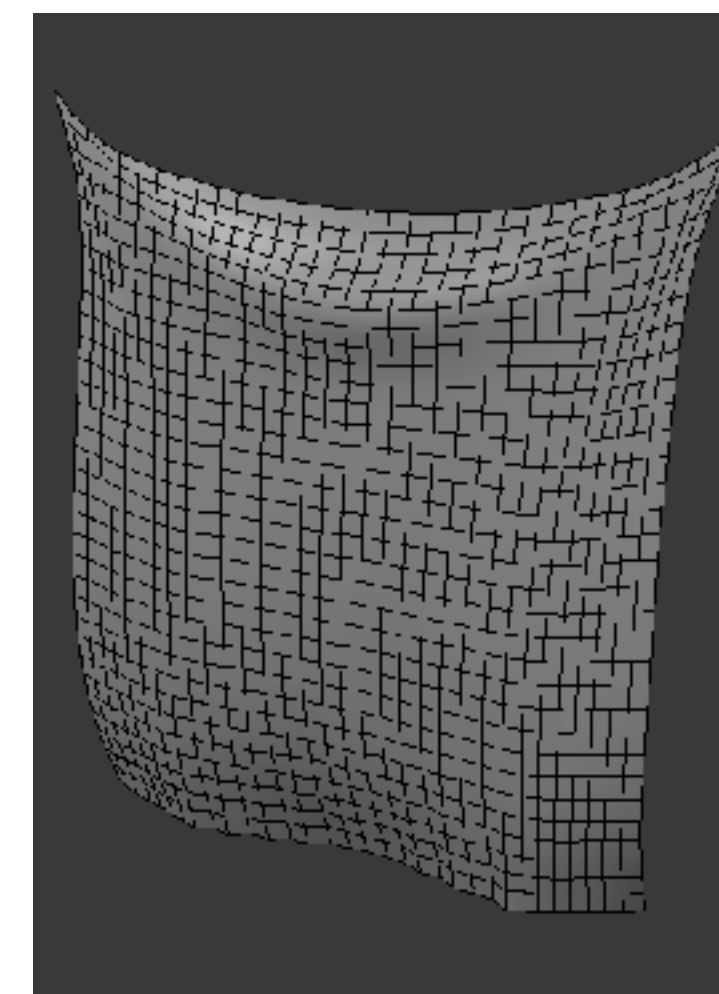
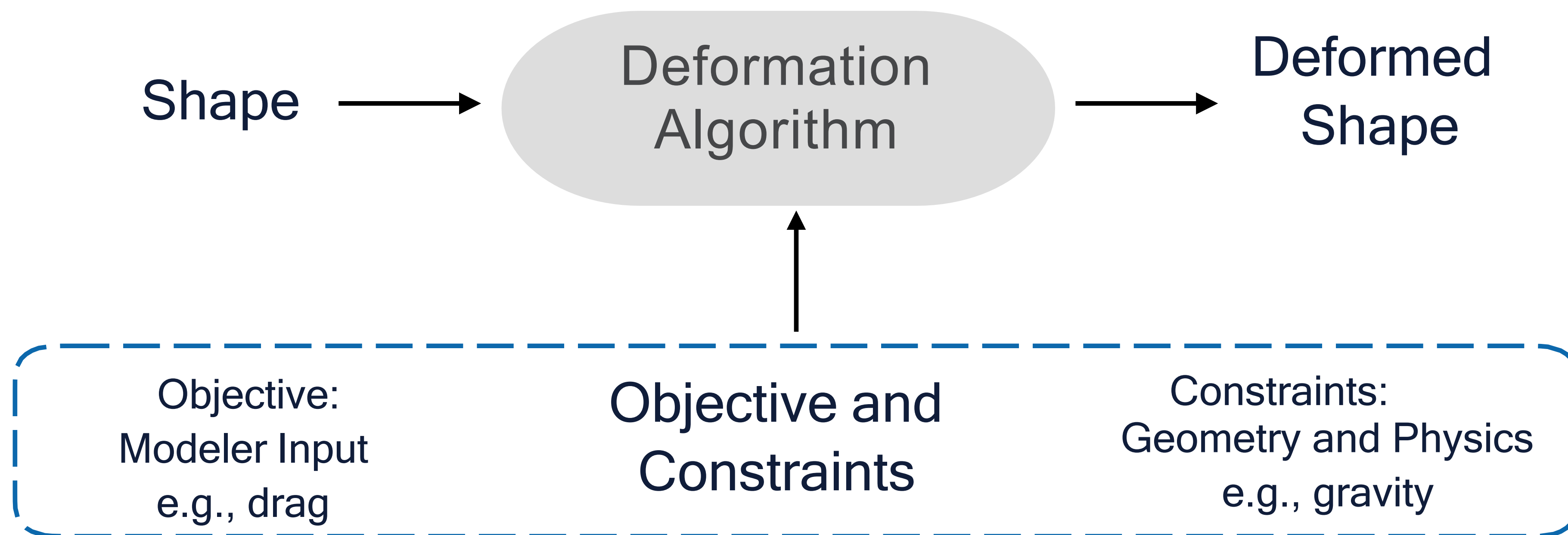
- Generate new shape by deforming an existing one
 - e.g., to create animate character motion



Shape Deformation

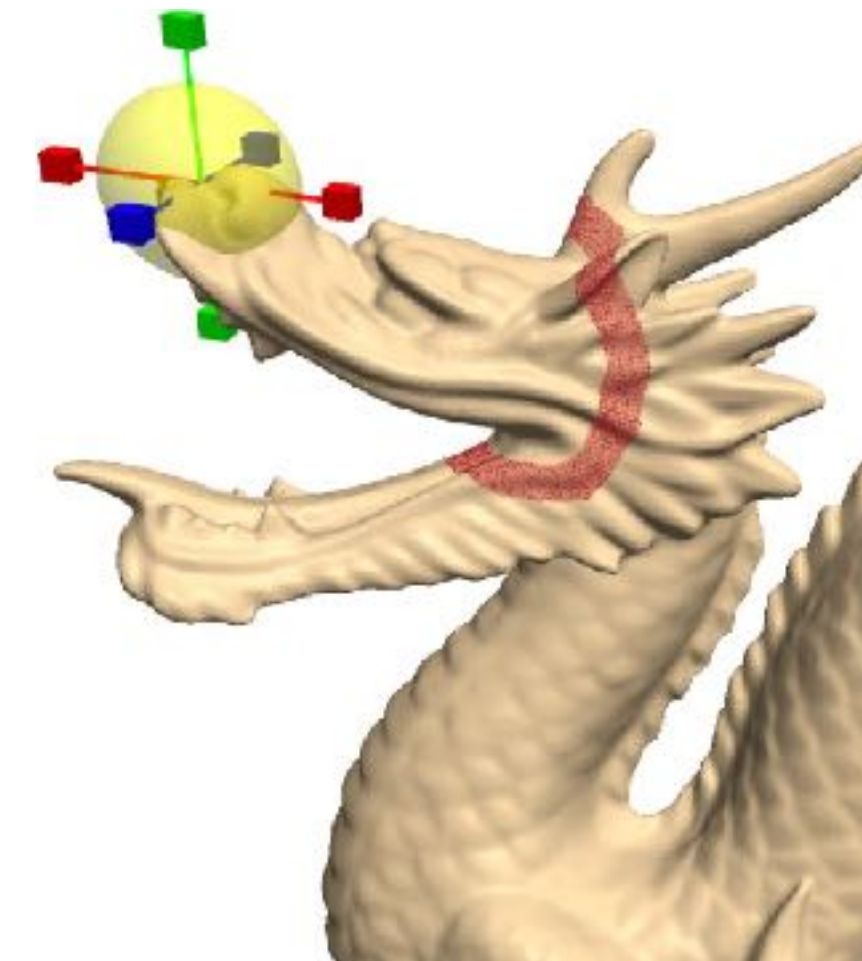
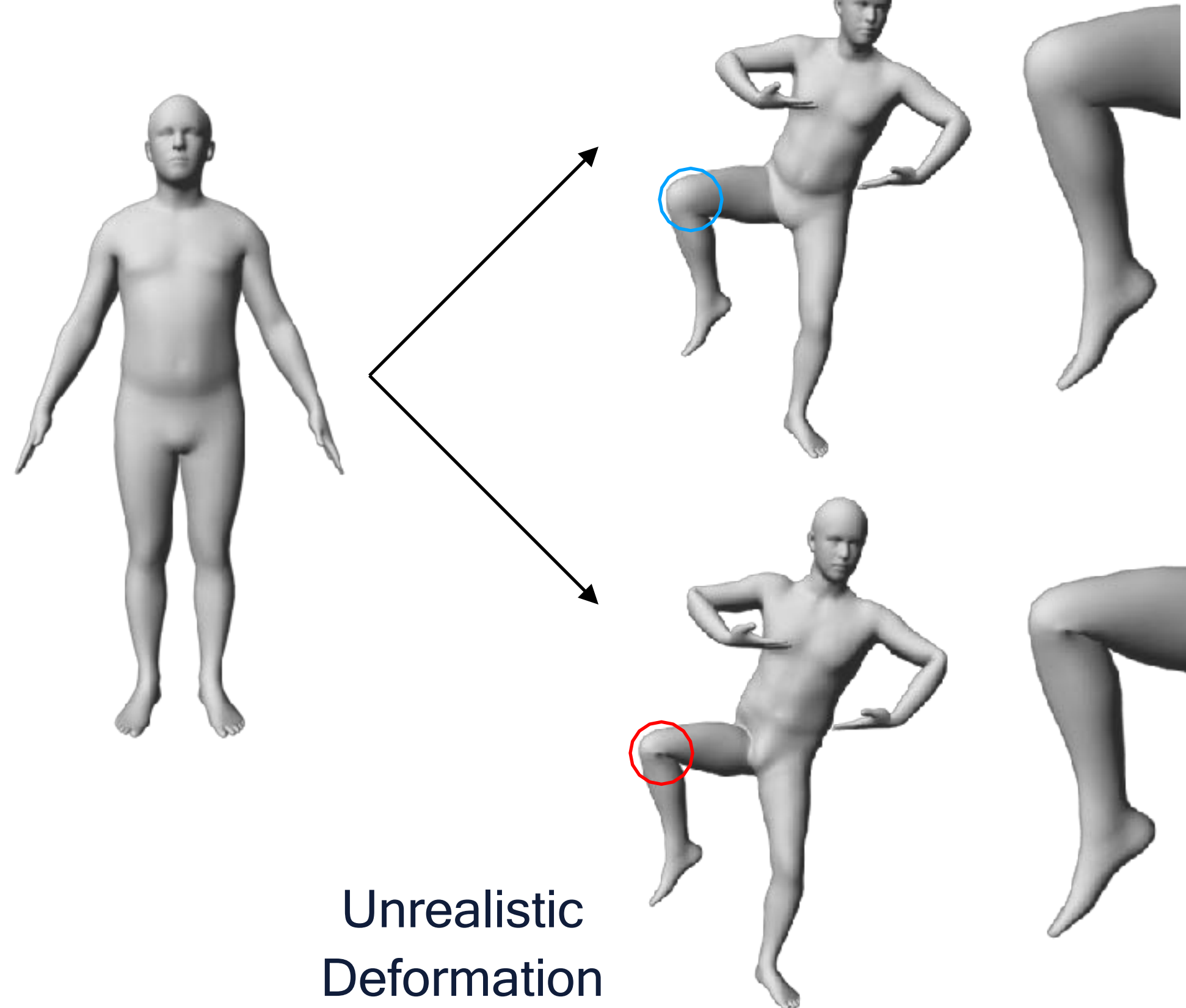


Shape Deformation



Preferred Deformation Algorithm

- Deformation should be natural
- Modeler works less, algorithm does more



Outline

- Introduction
- Surface Deformation
 - *Laplacian Surface Editing*
 - As-Rigid-As-Possible Deformation
- Space Deformation
- Skeleton Skinning

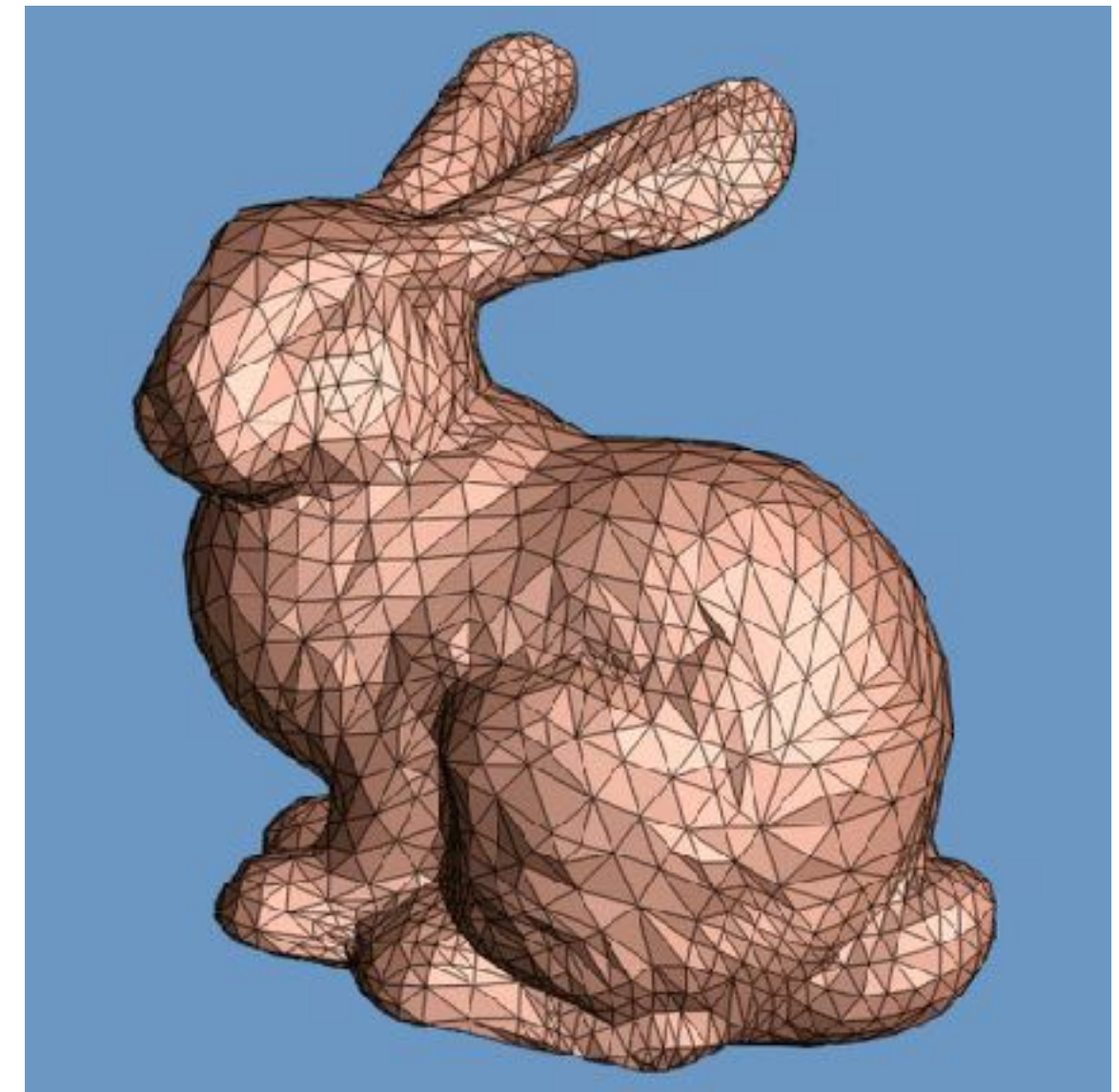
Shape Surface Representation

- Recall:
 - Piece-wise Linear Surface Representation
 - E.g., triangular mesh

$$V = v_1, v_2, \dots, v_n \subset \mathbb{R}^3$$

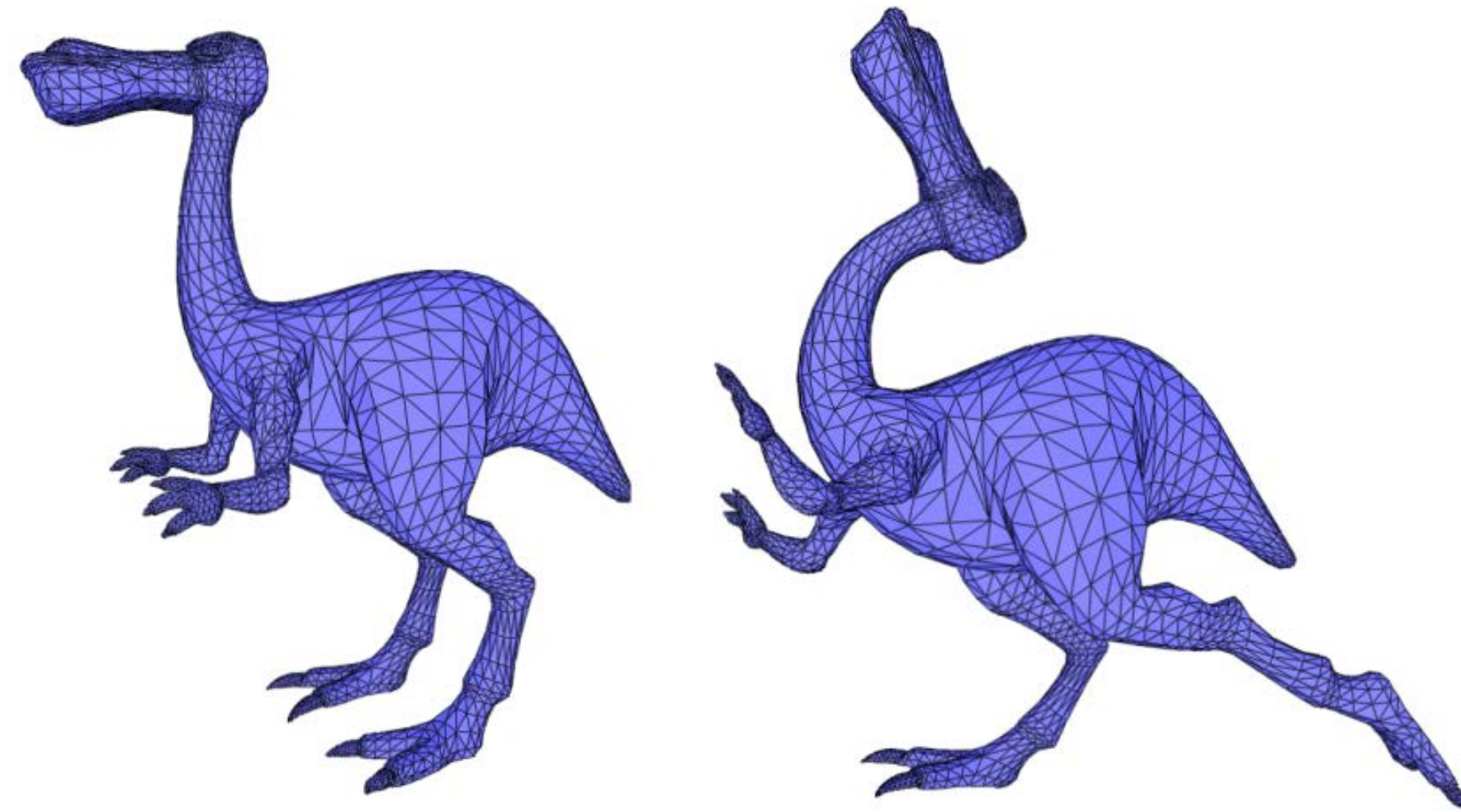
$$E = e_1, e_2, \dots, e_n \subseteq V \times V$$

$$F = f_1, f_2, \dots, f_n \subseteq V \times V \times V$$



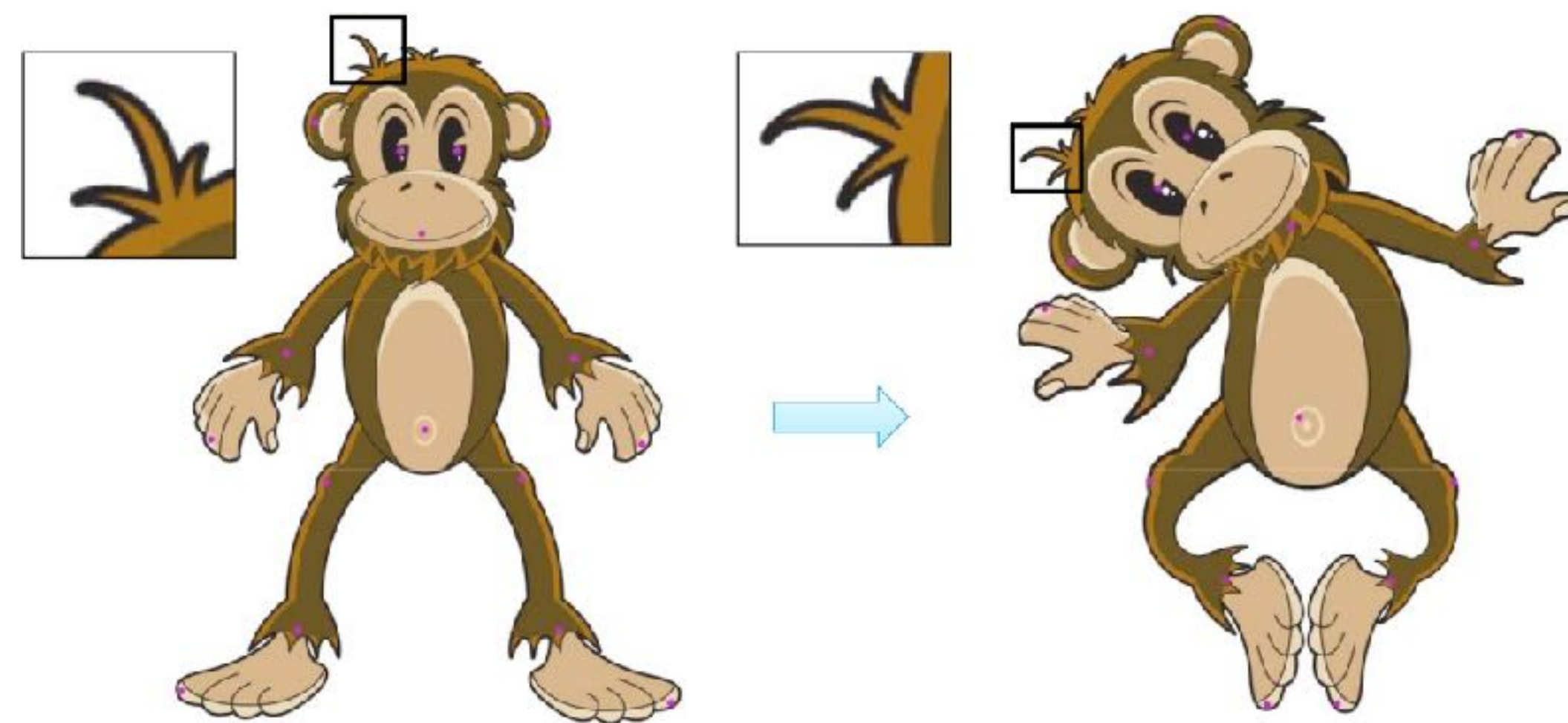
Surface Deformation

- Deformation is only defined on the surface
- Surface deformation: $d : V \rightarrow \mathbb{R}^3$
- V is vertices of mesh



Desired Surface Deformation

- Deformation is “natural”
 - It tries to preserve local geometry.



- Modelers do less, algorithms do more
 - E.g., given **vertex position objective** (new location of a few vertices), other points follow “naturally”.

How to Preserve Local Geometry?

- Recall: Curvature completely determines local surface geometry
- We want to find a “natural” deformation that preserves curvature

How to Preserve Curvature?

- Let us start with preserving mean curvature
- Laplacian can be used to approximate mean curvatures

The difference between a vertex x and the average position of its 1-ring neighborhood is a quantity that provides interesting geometric insight of the shape (see Figure 1). It can be shown that,

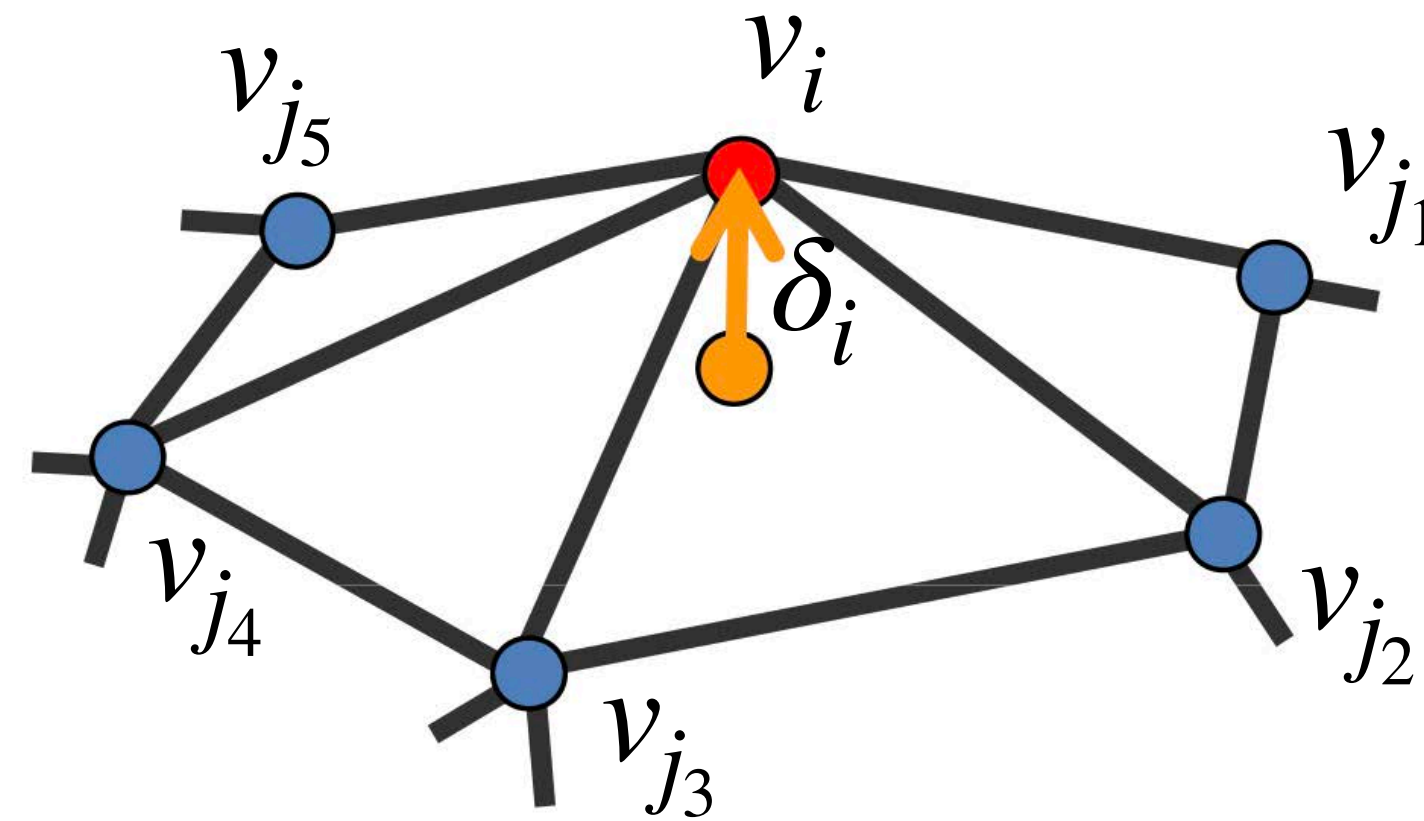
$$x - \frac{1}{|N(x)|} \sum_{y_i \in N(x)} y_i \approx H \vec{n} \Delta A \quad (2)$$

for a good mesh, where $N(x)$ is the 1-ring neighborhood vertices of x by the mesh topology, $H = \frac{1}{2}(\kappa_{min} + \kappa_{max})$ is the mean curvature at x (in the sense of the underlying continuous surface being approximated), $\vec{n}$ is the surface normal vector at x , and ΔA is a quantity proportional to the total area of the 1-ring fan (triangles formed by x and vertices along the 1-ring).

Laplacian Coordinates

- Different from common mesh representation in global coordinates, we can represent a point relative to its neighbors

- $$\delta_i = v_i - \sum_{j \in N(v_i)} \frac{1}{d_i} v_j$$
- d_i : degree of vertex i



Laplacian Coordinates

- Recall:

Laplacian matrix: $L = D - A$

D is degree matrix and A is adjacency matrix

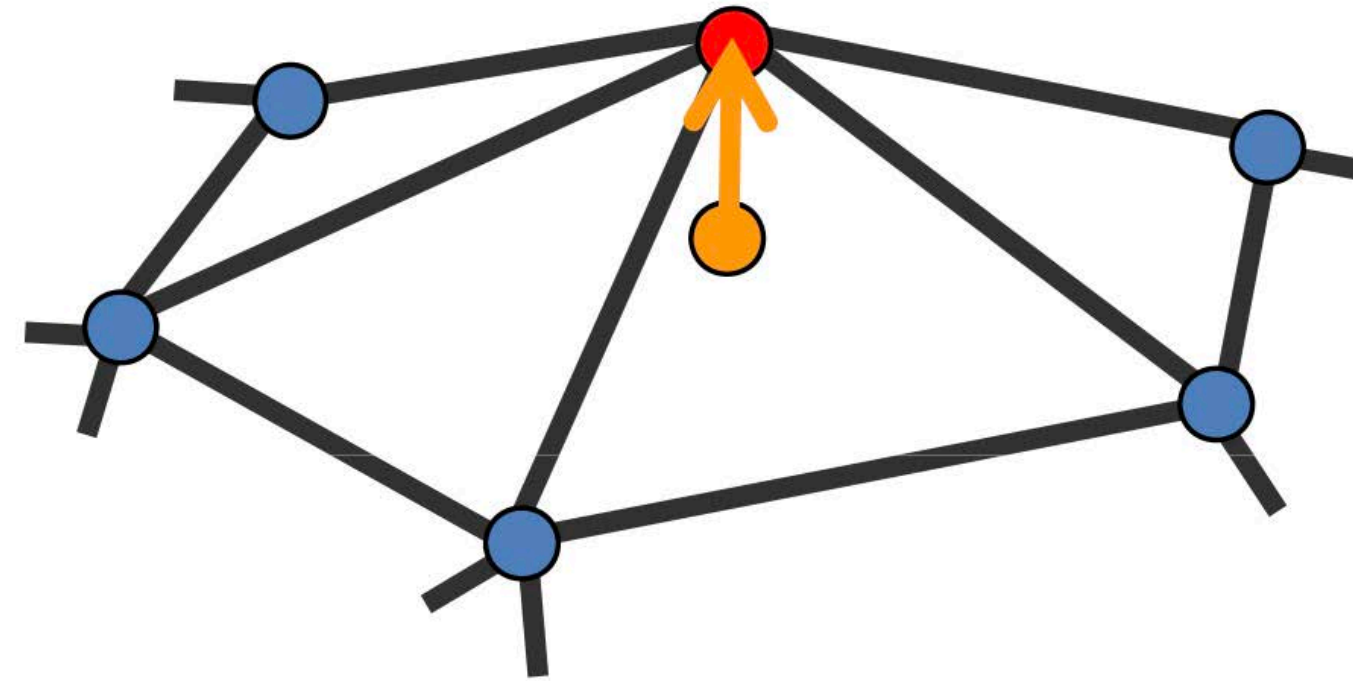
- Differential coordinates can be computed by a **normalized laplacian matrix**

$$\delta_i = v_i - \sum_{j \in N(i)} \frac{1}{d_i} v_j$$

$$\delta = (I - D^{-1}A)V = (D^{-1}L)V$$

- V is a $n \times 3$ matrix denotes vertices position

Laplacian Coordinates Property



- Direction of δ_i approximates the **normal direction**
- Size of the δ_i approximates the **mean curvature**

$$\delta_i = v_i - \sum_{j \in N(i)} \frac{1}{d_i} v_j$$

- Note: mean curvature cannot fully determine local geometry. 2 numbers are needed

Deform by Laplacian Coordinates

- Input: vertex (control point) position objective
- Consider a simple objective of moving several vertices towards the new location: $v'_i = u_i$, where v' is vertex after deformation.
- Energy function:

$\mathcal{L}(v'_i)$ is laplacian coordinates of v'_i

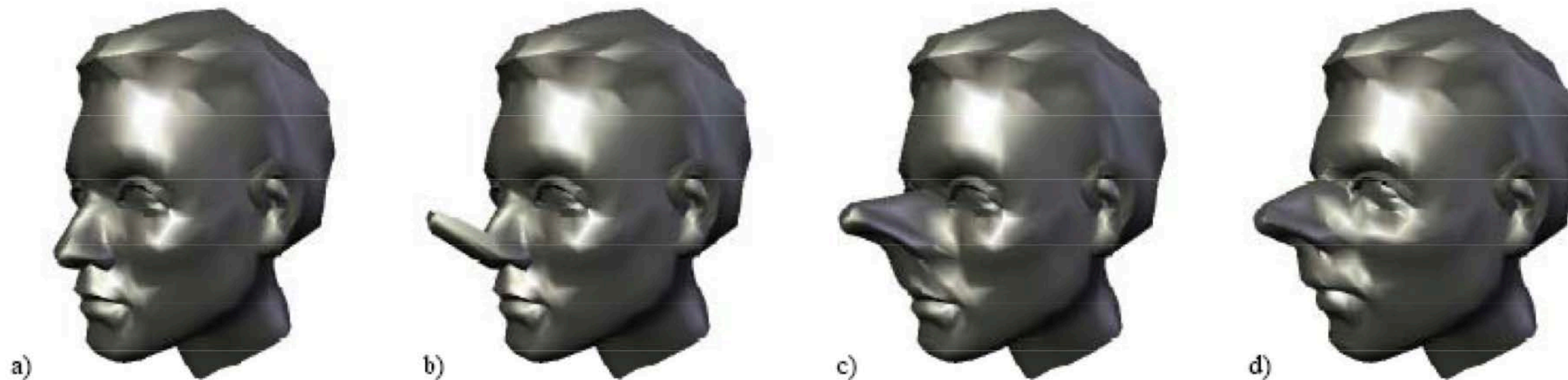
$$E(V') = \sum_{i=1}^n \|\delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$

Laplacian Coordinate of Original Mesh Laplacian Coordinate of Deformed Mesh Objective: Control Point

Deform by Laplacian Coordinates

- Deformed shape can be solved by minimizing $E(V')$

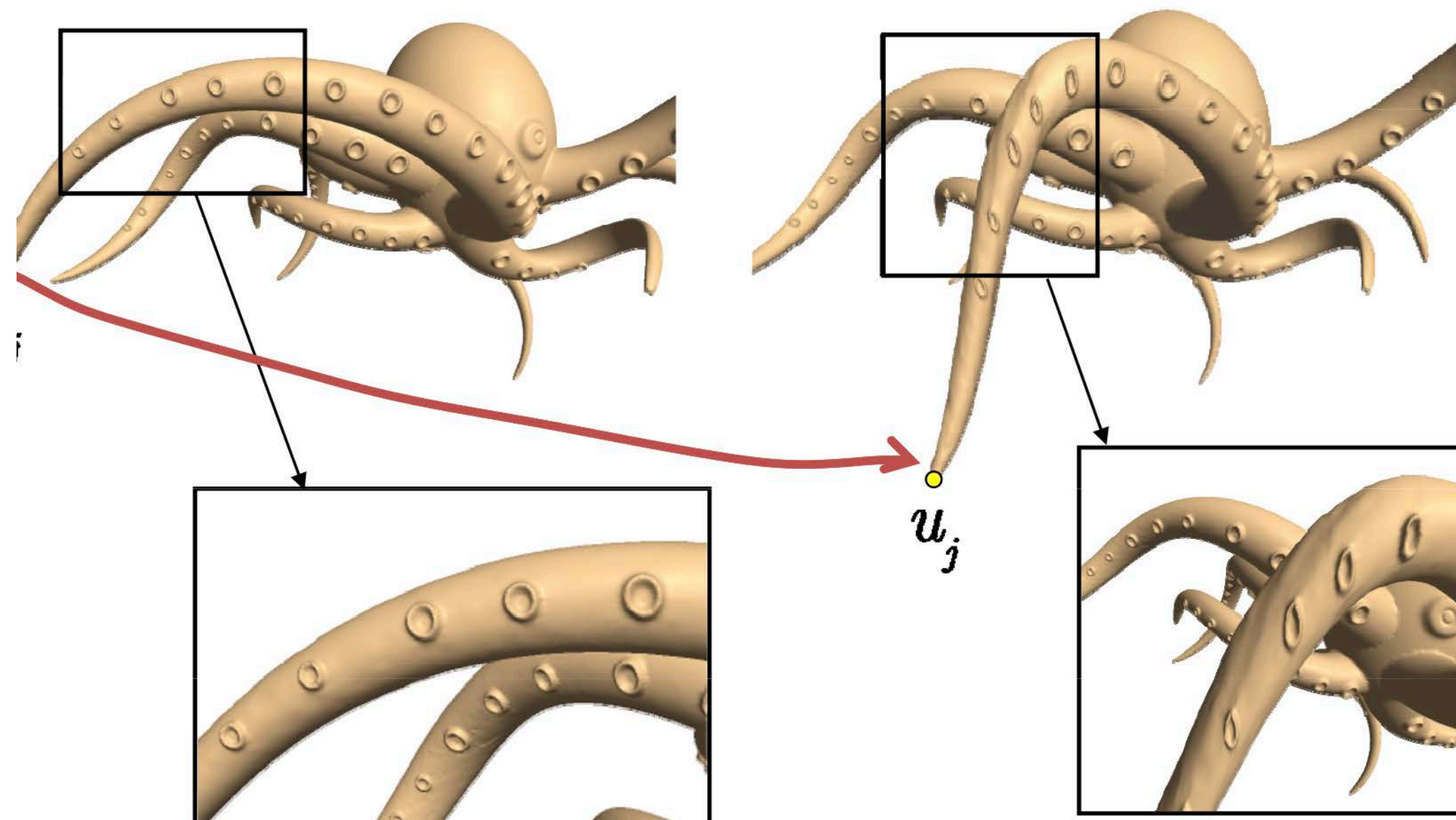
$$E(V') = \sum_{i=1}^n \|\delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$



$E(V')$ decreases by iterations

Issues?

- Other than preserving the mean curvature, $\sum_{i=1}^n \|\delta_i - \mathcal{L}(v'_i)\|^2$ also have tried to preserve normal.
- However, normal preservation is undesired
- How can we cancel the effect of normal preservation?



Laplacian Coordinates Under Transformation

- Normals are invariant under translation, so

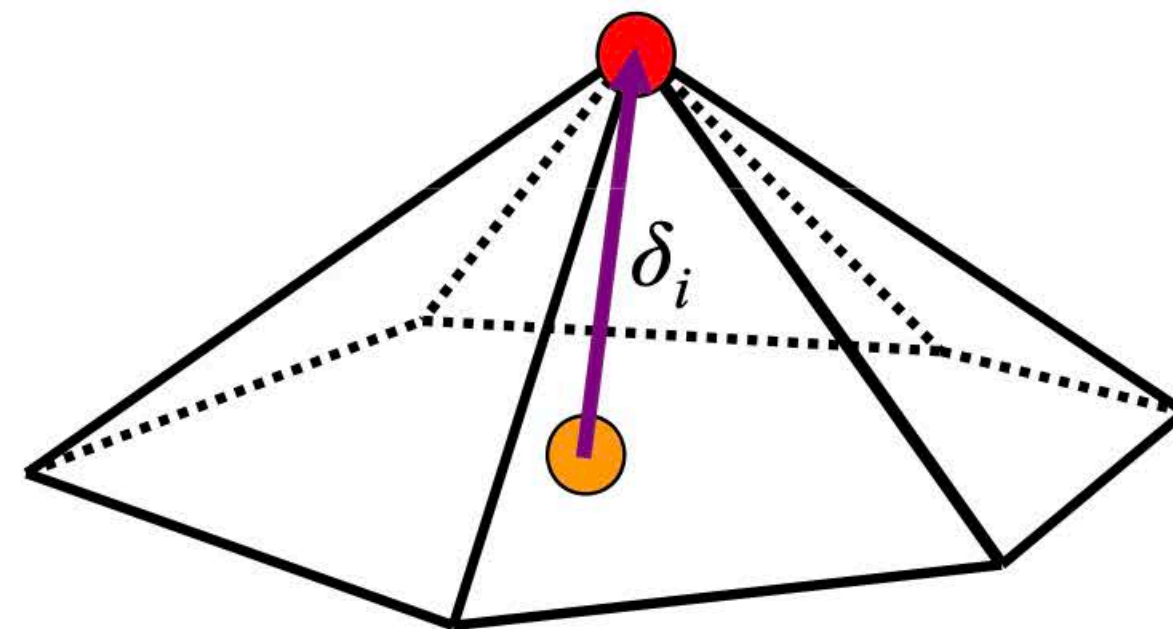
$$\mathcal{L}(v_i) = \mathcal{L}(v_i + t)$$

Laplacian Coordinates Under Transformation

- Normals are invariant under translation, so

$$\mathcal{L}(v_i) = \mathcal{L}(v_i + t)$$

- However, normal changes under rotation, so Laplacian coordinates change under rotation

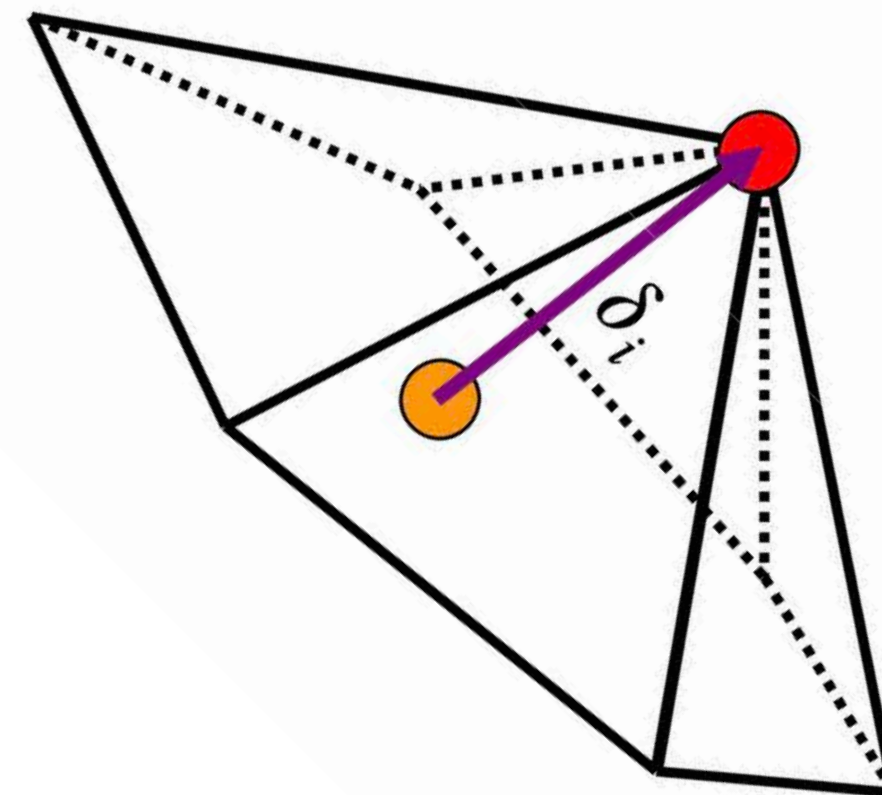


Laplacian Coordinates Under Transformation

- Normals are invariant under translation, so

$$\mathcal{L}(v_i) = \mathcal{L}(v_i + t)$$

- However, normal changes under rotation, so Laplacian coordinates change under rotation



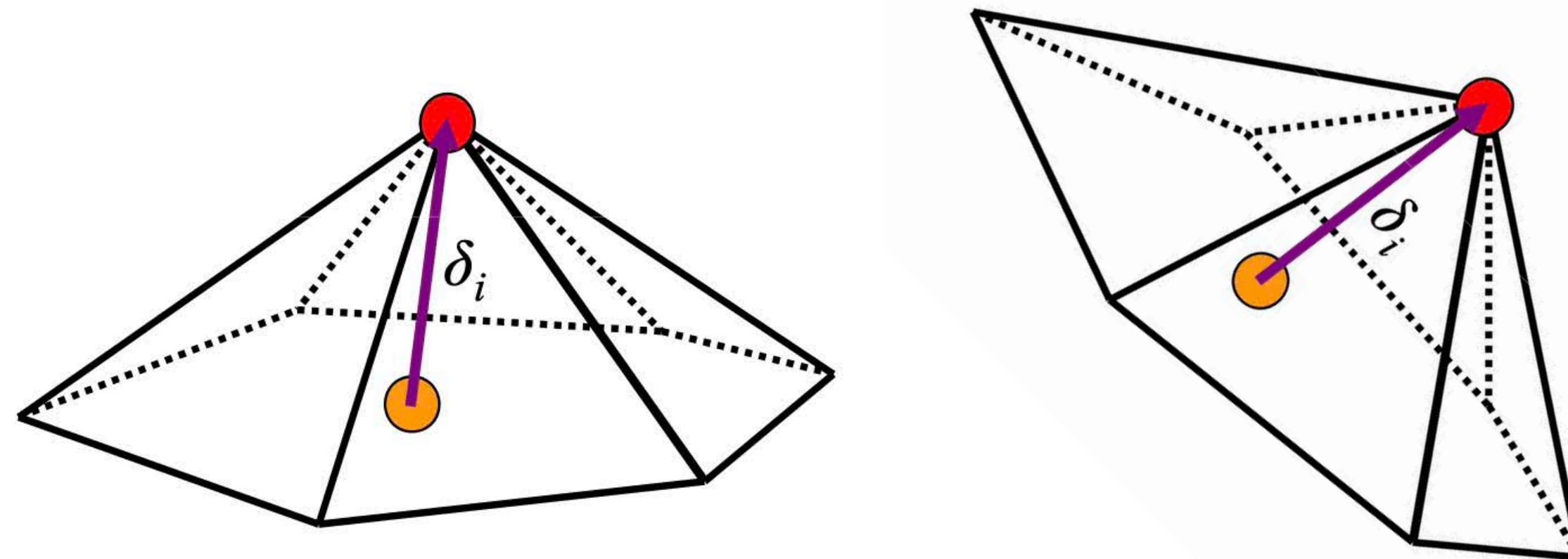
Laplacian Coordinates Under Transformation

- Normals are invariant under translation, so

$$\mathcal{L}(v_i) = \mathcal{L}(v_i + t)$$

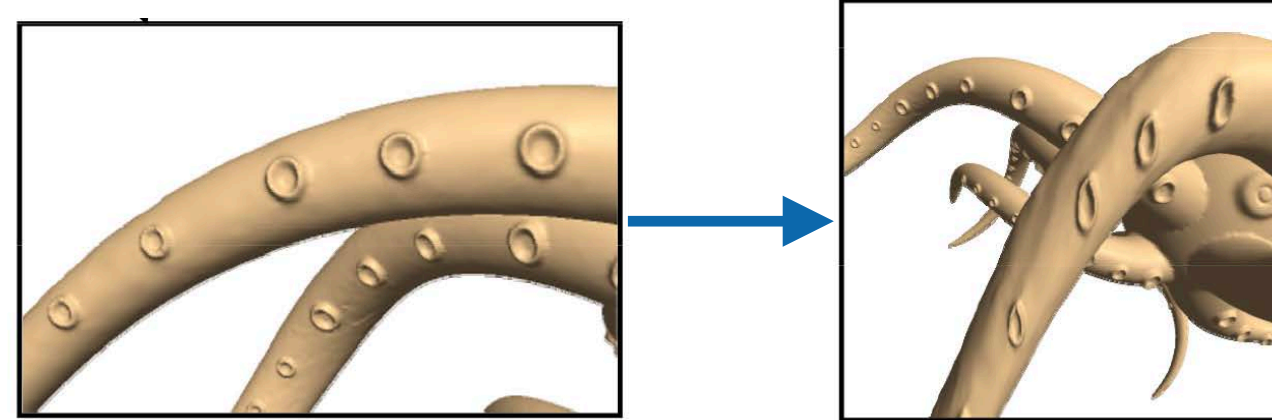
- However, normal changes under rotation, so Laplacian coordinates change under rotation

$$R\mathcal{L}(v_i) = \mathcal{L}(Rv_i)$$



Solution

- After deformation, assuming that the local region of the surface will rotate



- Original optimization target:

$$E(V') = \sum_{i=1}^n \|\delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$

- New optimization target by introducing a variable to cancel the rotation:

$$E(V') = \min_{\{R_i\}} \sum_{i=1}^n \|\mathbf{T}_i \delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$

Alternating Optimization

We optimize vertex V and transformation T iteratively

1. Estimate shape V' given transformation

$$\min_{V'} E(V') = \sum_{i=1}^n \|T_i \delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$

2. Estimate transformation T_i from the deformed shape

$$\min_{T_i} \left(\|T_i \mathbf{v}_i - \mathbf{v}'_i\|^2 + \sum_{j \in \mathcal{N}_i} \|T_i \mathbf{v}_j - \mathbf{v}'_j\|^2 \right)$$

Alternating Optimization

We optimize vertex V and transformation T iteratively

1. Estimate shape V' given transformation

$$\min_{V'} E(V') = \sum_{i=1}^n \|T_i \delta_i - \mathcal{L}(v'_i)\|^2 + \sum_{i=m}^n \|v'_i - u_i\|^2$$

Unconstrained least square

2. Estimate transformation T_i from the deformed shape

$$\min_{T_i} \left(\|T_i \mathbf{v}_i - \mathbf{v}'_i\|^2 + \sum_{j \in \mathcal{N}_i} \|T_i \mathbf{v}_j - \mathbf{v}'_j\|^2 \right)$$

Recall: Umeyama Algorithm for similarity transformation

Other Issues?

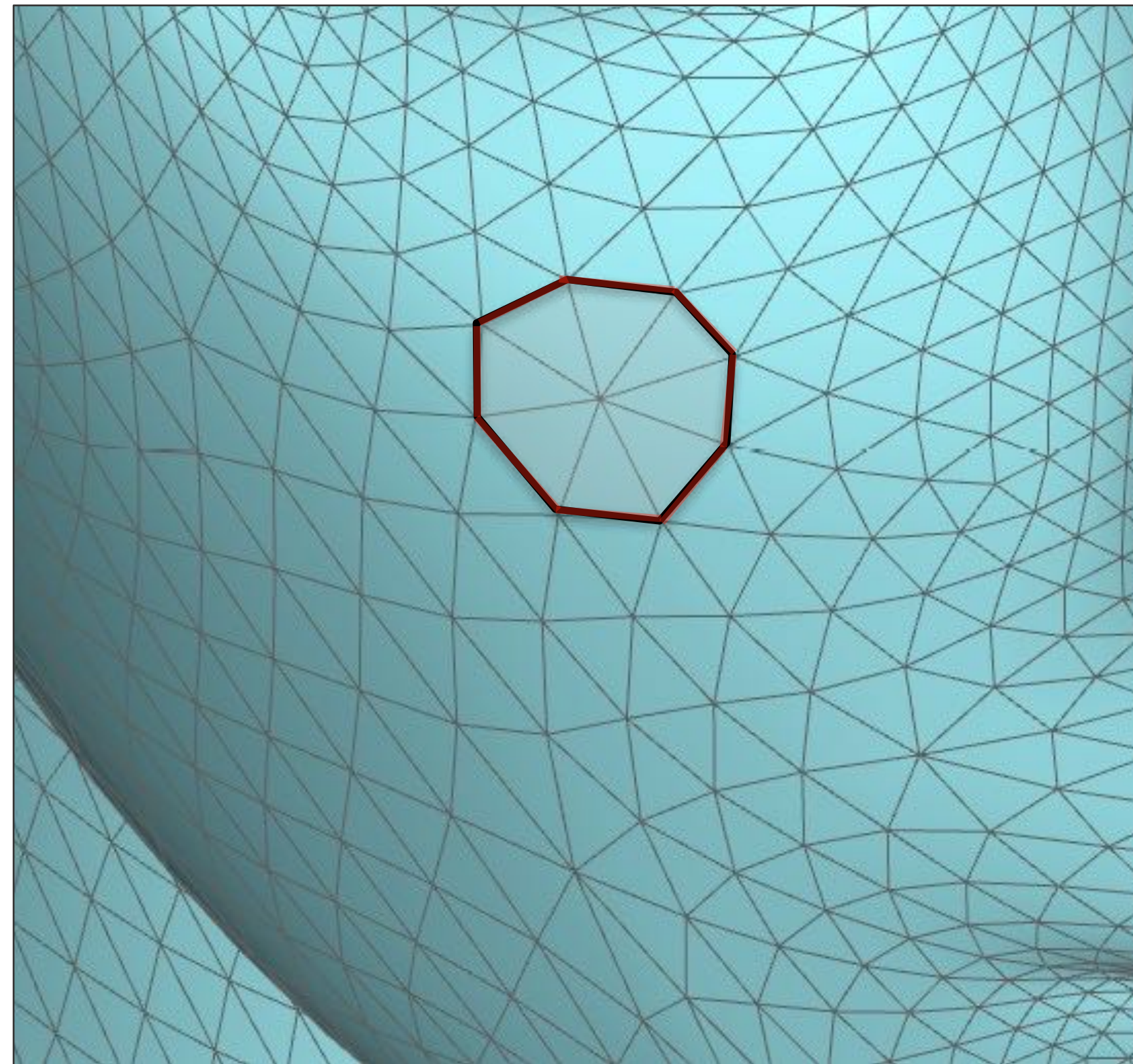
- Only mean curvature is considered
 - Full curvature (2 numbers) is required to fully determine the local geometry.
 - Lengths and angles are required for global geometry
- Solution to the issue
 - Deformation energy should consider both mean curvature and Gaussian curvature
 - Deformation energy should preserve lengths and angles

Outline

- Introduction
- Surface Deformation
 - Laplacian Surface Editing
 - *As-Rigid-As-Possible Deformation*
- Space Deformation
- Skeleton Skinning

Local Deformation

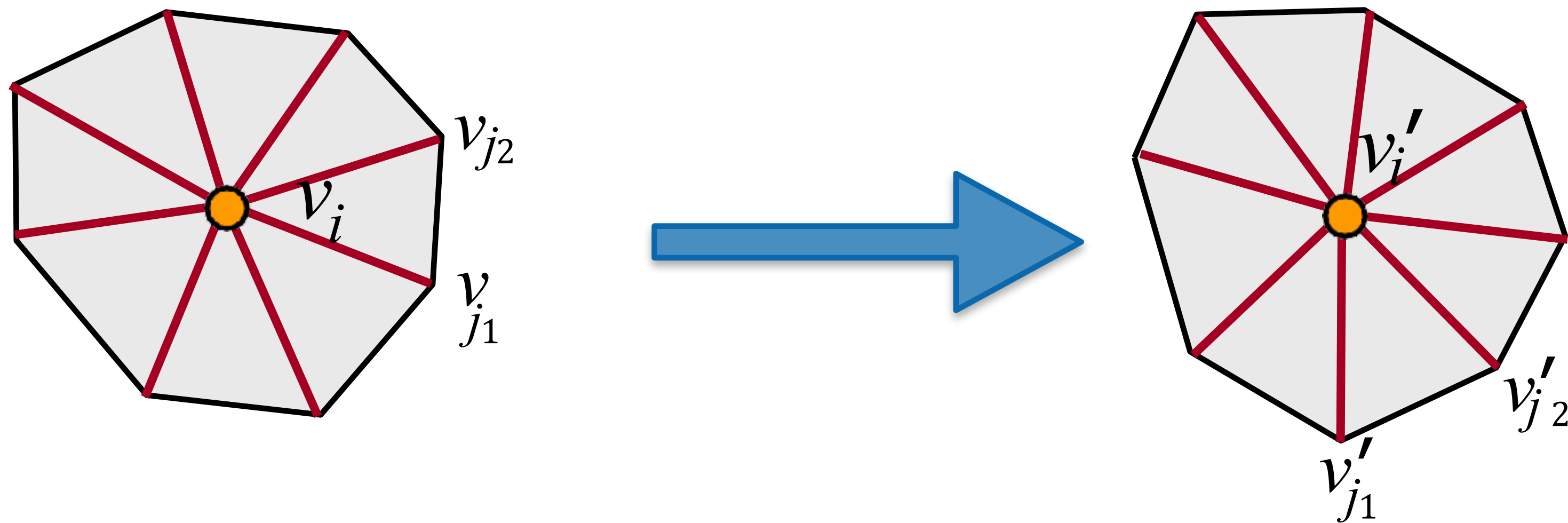
- Let's look at a local region centered at a vertex (called a cell).



How do we define a better deformation energy of this cell?

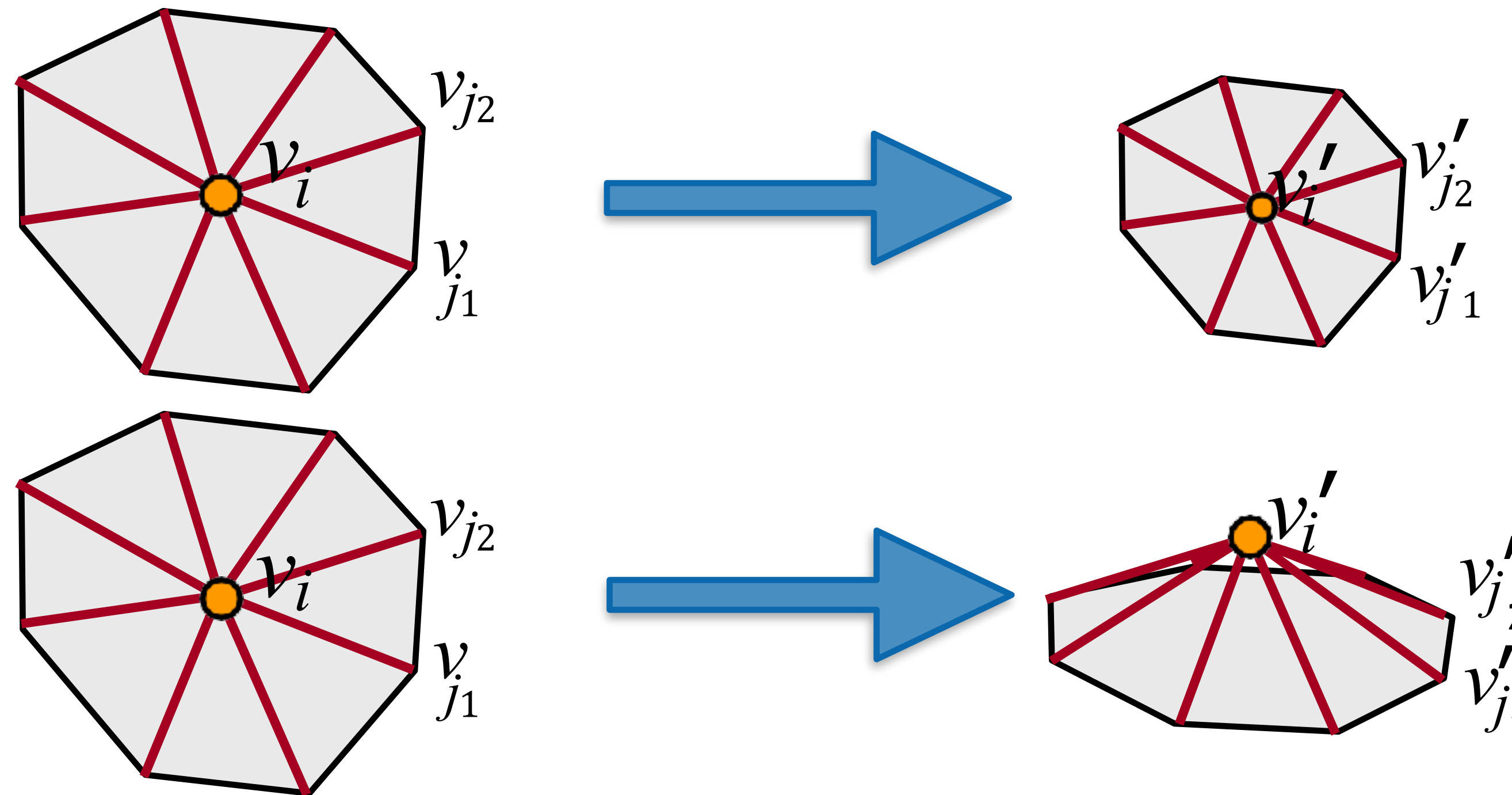
Desired Property for Deformation Energy

- Translation and rotation should not change the deformation energy.



Desired Property for Deformation Energy

- Translation and rotation should not change the deformation energy.
- **Stretching** (length change) and **bending** (angle change) increase deformation energy.



Local Deformation Energy

- Translation and rotation should not change the deformation energy.
- **Stretching** (length change) and **bending** (angle change) increase deformation energy.

$$E(v_i) = \min_{R_i} \sum_{j \in N(i)} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$

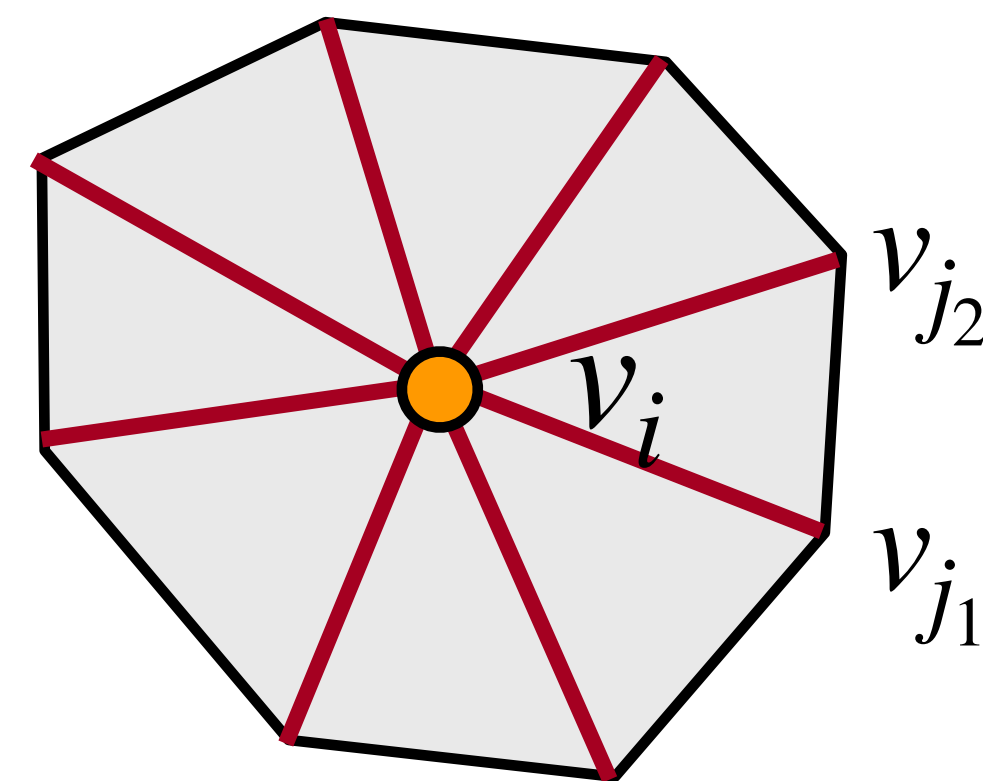
Local Deformation Energy

Local Deformation Energy

- Translation and rotation should not change the deformation energy.
- **Stretching** (length change) and **bending** (angle change) increase deformation energy.

Minimum over all rotations, rotation-invariant

$$E(v_i) = \min_{R_i} \sum_{j \in N(i)} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$

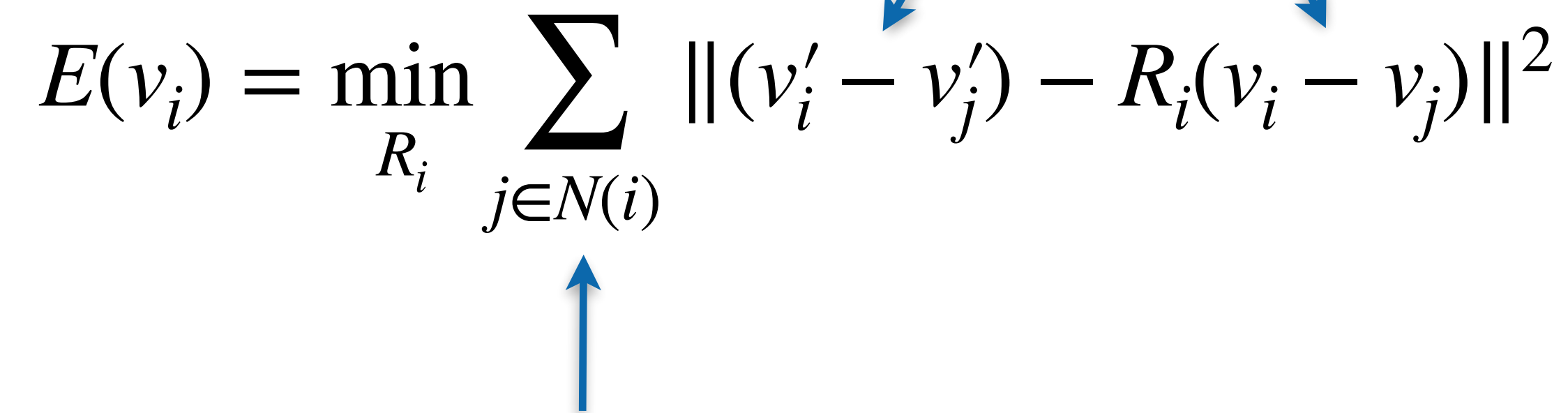


Relative to the cell center (v_i or v'_i), translation-invariant

Local Deformation Energy

- Translation and rotation should not change the deformation energy.
- **Stretching** (length change) and **bending** (angle change) increase deformation energy.

Penalize change of length

$$E(v_i) = \min_{R_i} \sum_{j \in N(i)} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$


R_i is shared by the cell, penalize change of angle

Local Deformation Energy

$$E(v_i) = \min_{R_i} \sum_{j \in N(i)} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$

- It is (again) an Orthogonal Procrustes Problem!

Total Deformation Energy

- Sum up the local deformation energy over all vertices

$$E(V') = \min_{v'} \sum_{i=1}^n \min_{R_i} \sum_{j \in N(i)} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$

$s.t. \quad v'_j = c_j, j \in C$

C : the set of control point indices

- Minimizing total deformation energy
 - As-Rigid-As-Possible deformation (ARAP deformation)

Total Deformation Energy

- Alternating optimization
 - Given initial guess v'_0 , find optimal rotations R_i .
 - This is a per-cell task! We already showed how to estimate R_i when v, v' are known

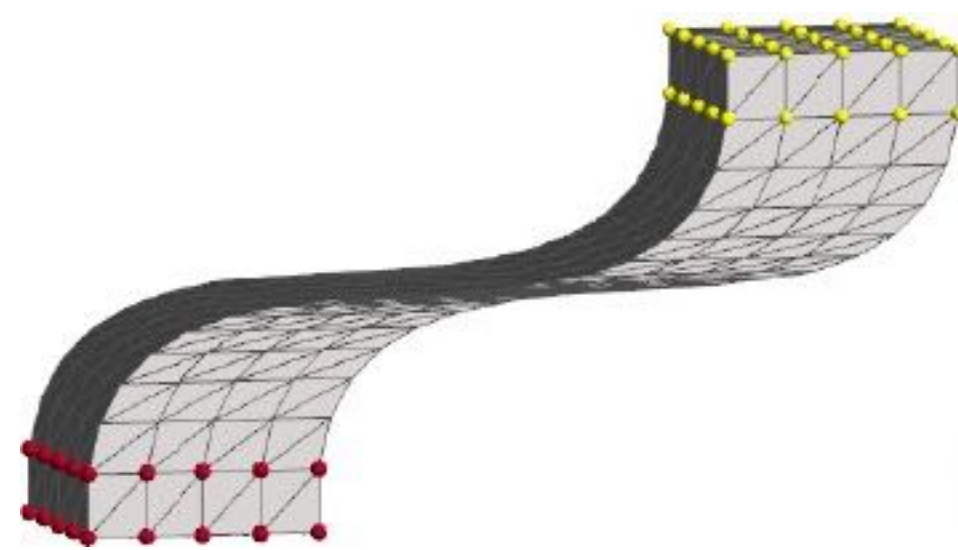
- Given the R_i (fixed), minimize the energy by finding new v'

$$E(V') = \min_{v'} \sum_{i=1}^n \sum_{j \in N(i)} \|(\textcolor{blue}{v'_i} - \textcolor{blue}{v'_j}) - \textcolor{red}{R_i}(\textcolor{red}{v_i} - \textcolor{red}{v_j})\|^2$$

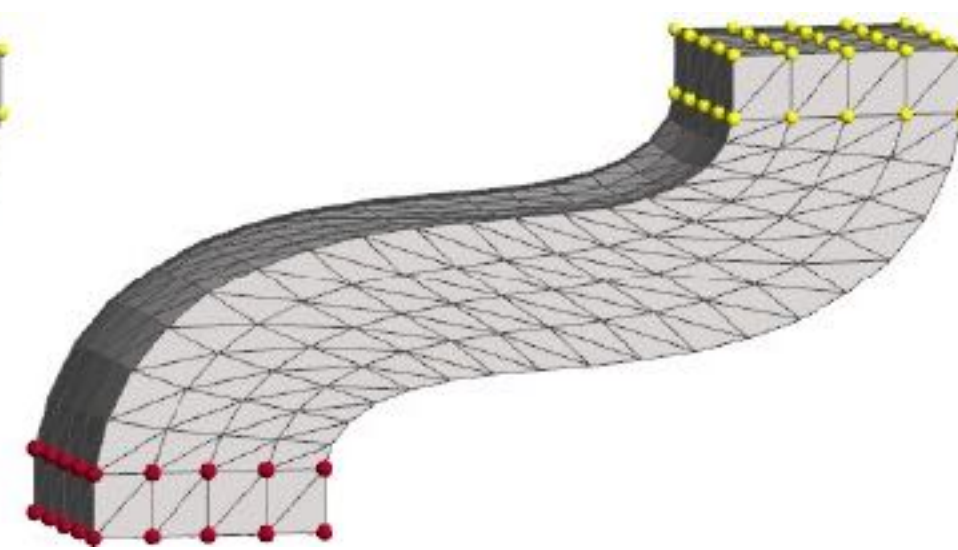
Linear Least Square

Initialization

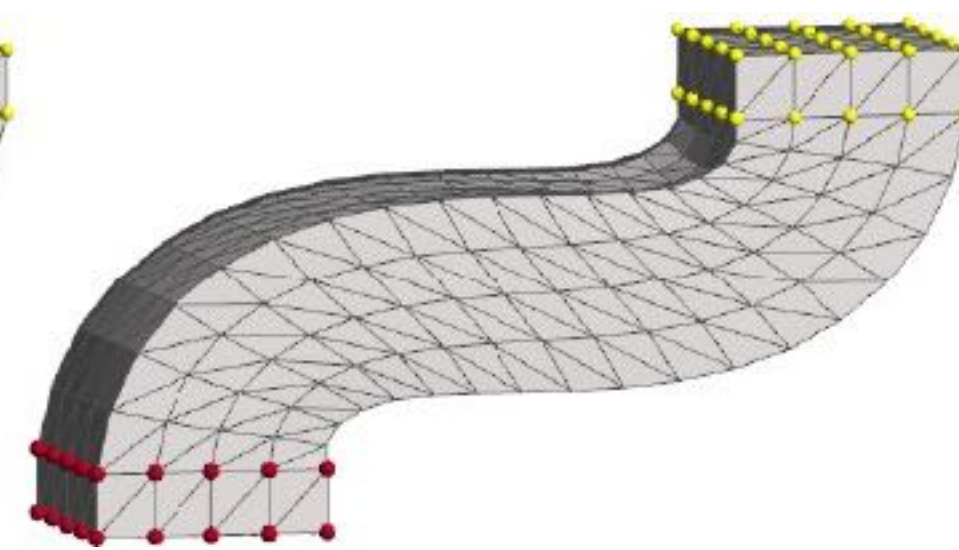
Start from naïve Laplacian editing as initial guess and iterate



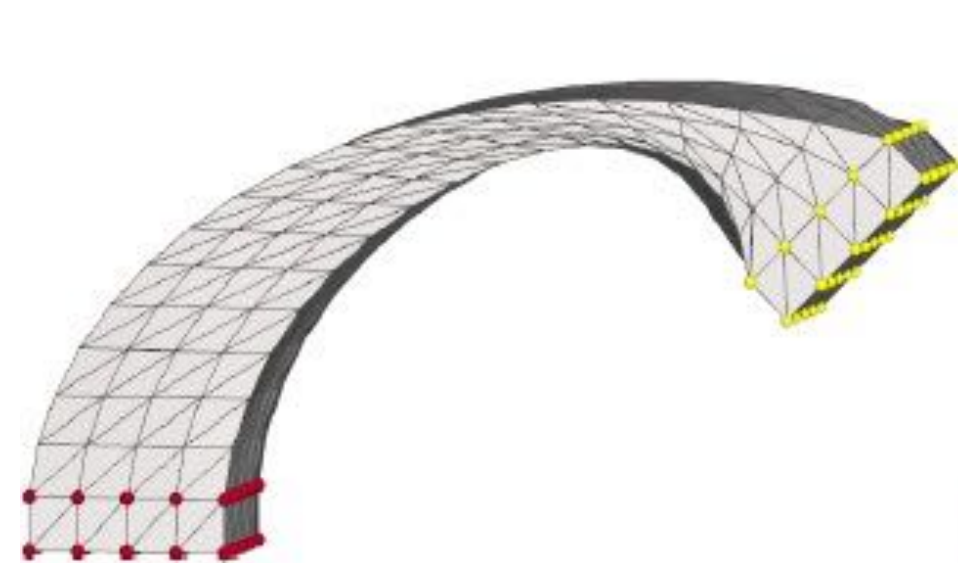
initial guess



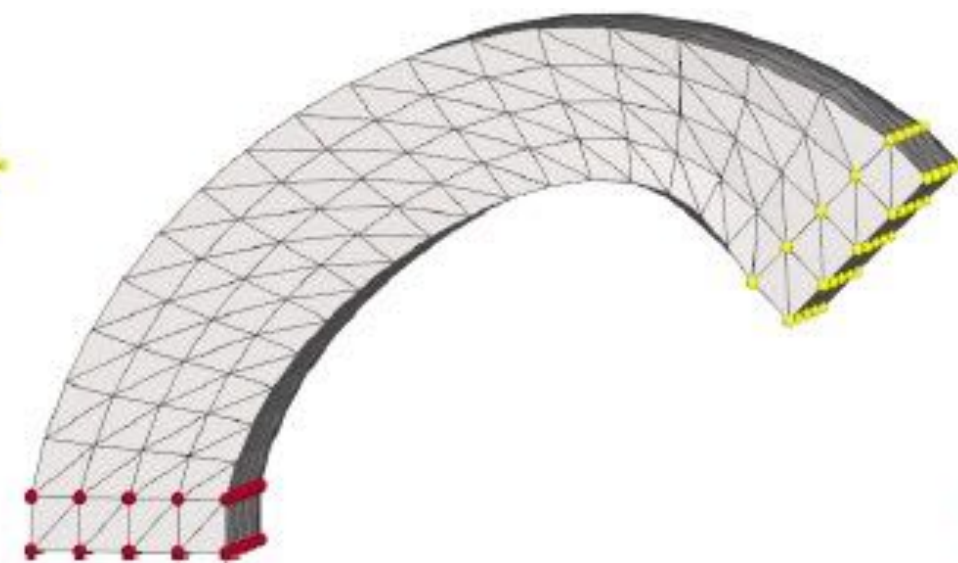
1 iteration



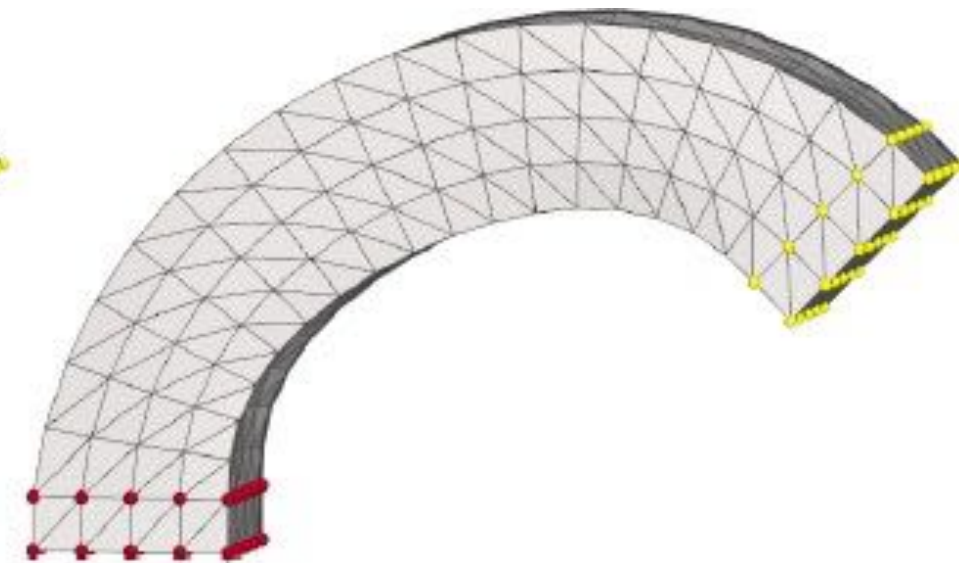
2 iterations



initial guess

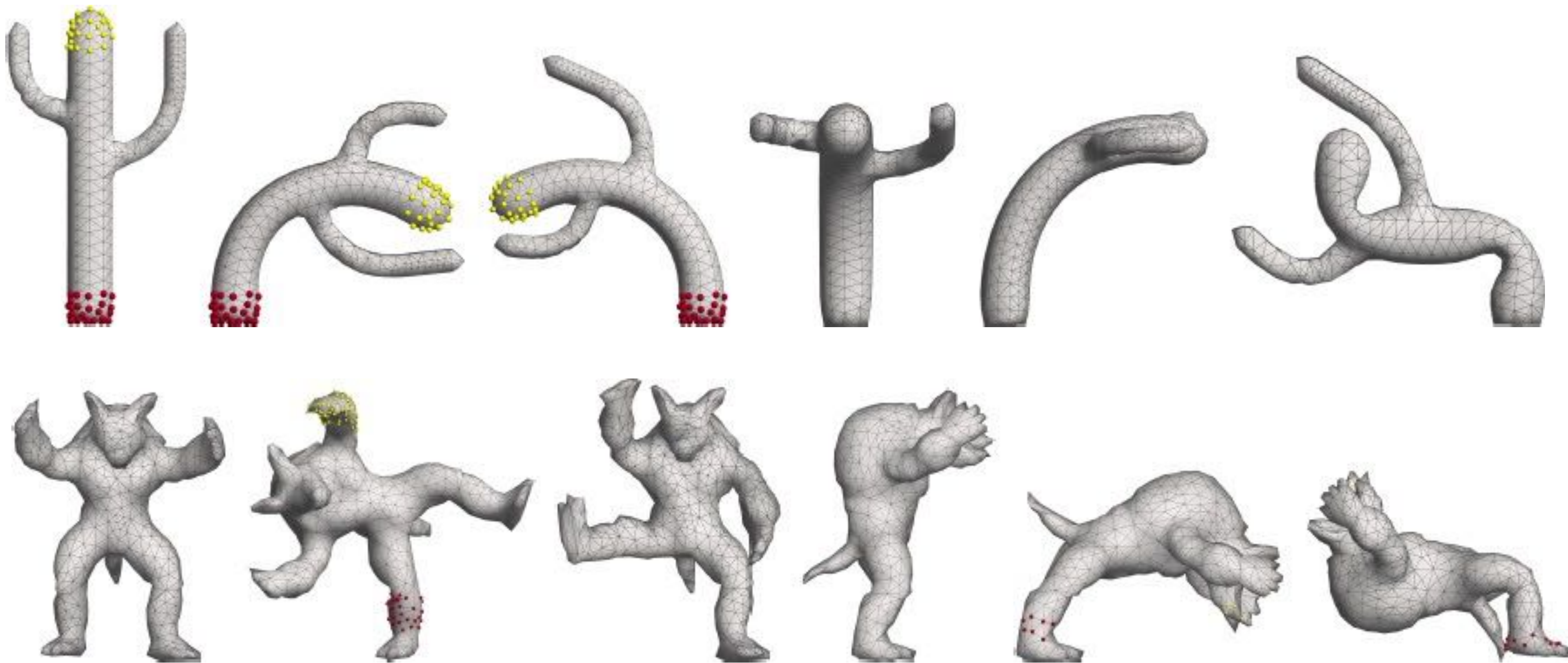


1 iterations



4 iterations

Examples



Summary of ARAP

- As-rigid-as-possible deformation iteratively minimize the deformation energy.
- The deformation energy penalizes both mean curvature change and length change.

Further Comments

- Iterative algorithm, slow on large meshes.
- Guaranteed to converge (energy is bounded and monotonically decreasing for each iteration)
- The idea can generalize to other energy definition or 3D volume deformation (real physical deformation)

Outline

- Introduction
- Surface Deformation
- *Space Deformation*
- Skeleton Skinning

Surface vs Space Deformation

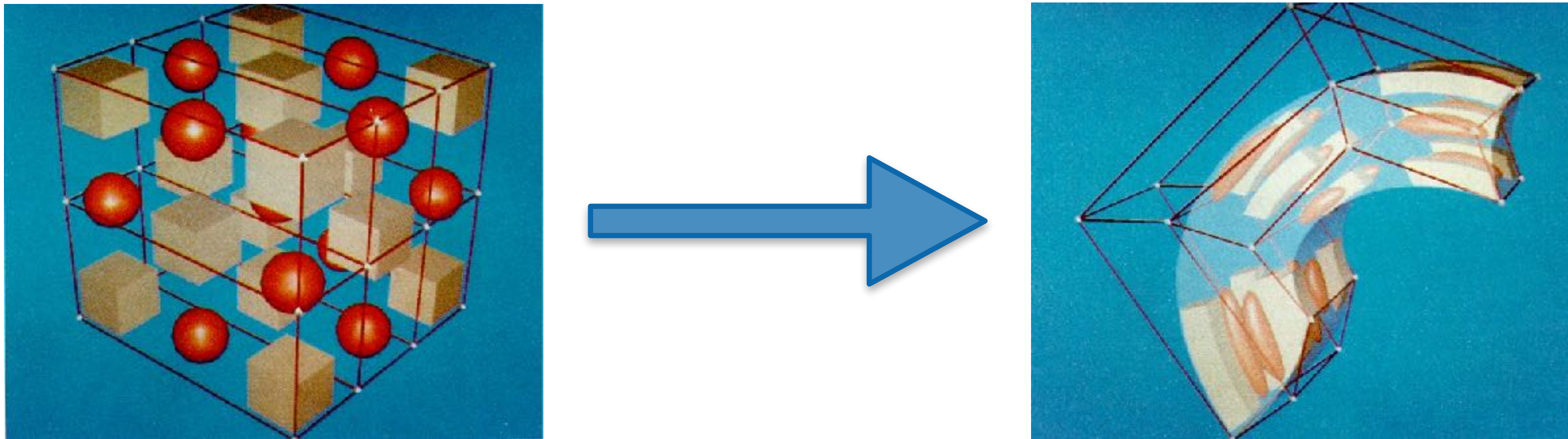
- Previously: surface deformation
 - Move vertices of the mesh
- Space deformation
 - Define a function that warps the $\mathbb{R}^3$ space.

$$f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$$

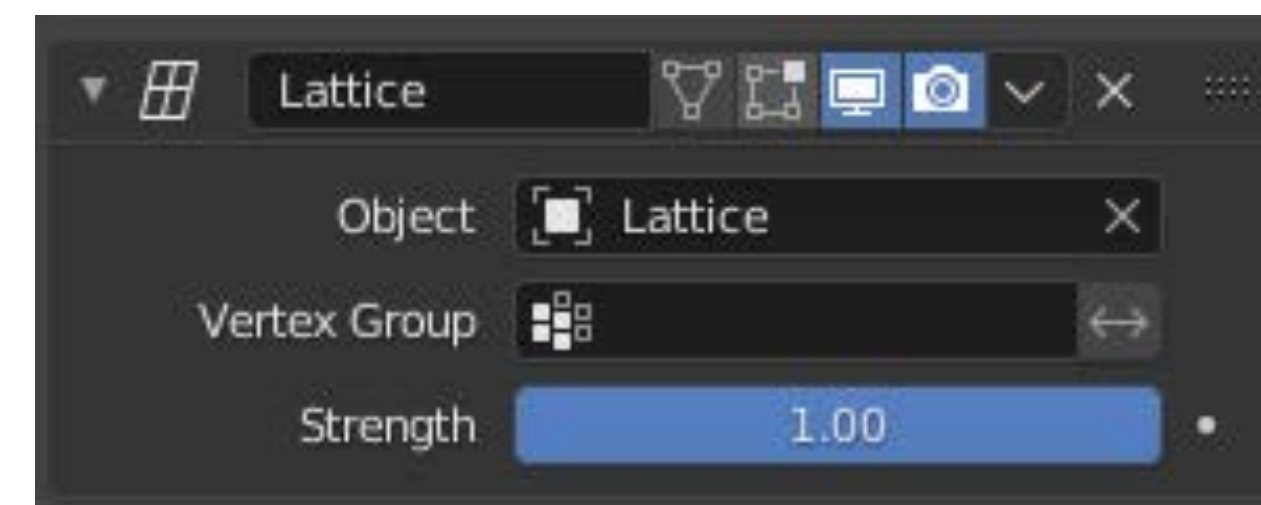
- Evaluate the space deformation on mesh vertices to deform the mesh.

Free-Form Deformation

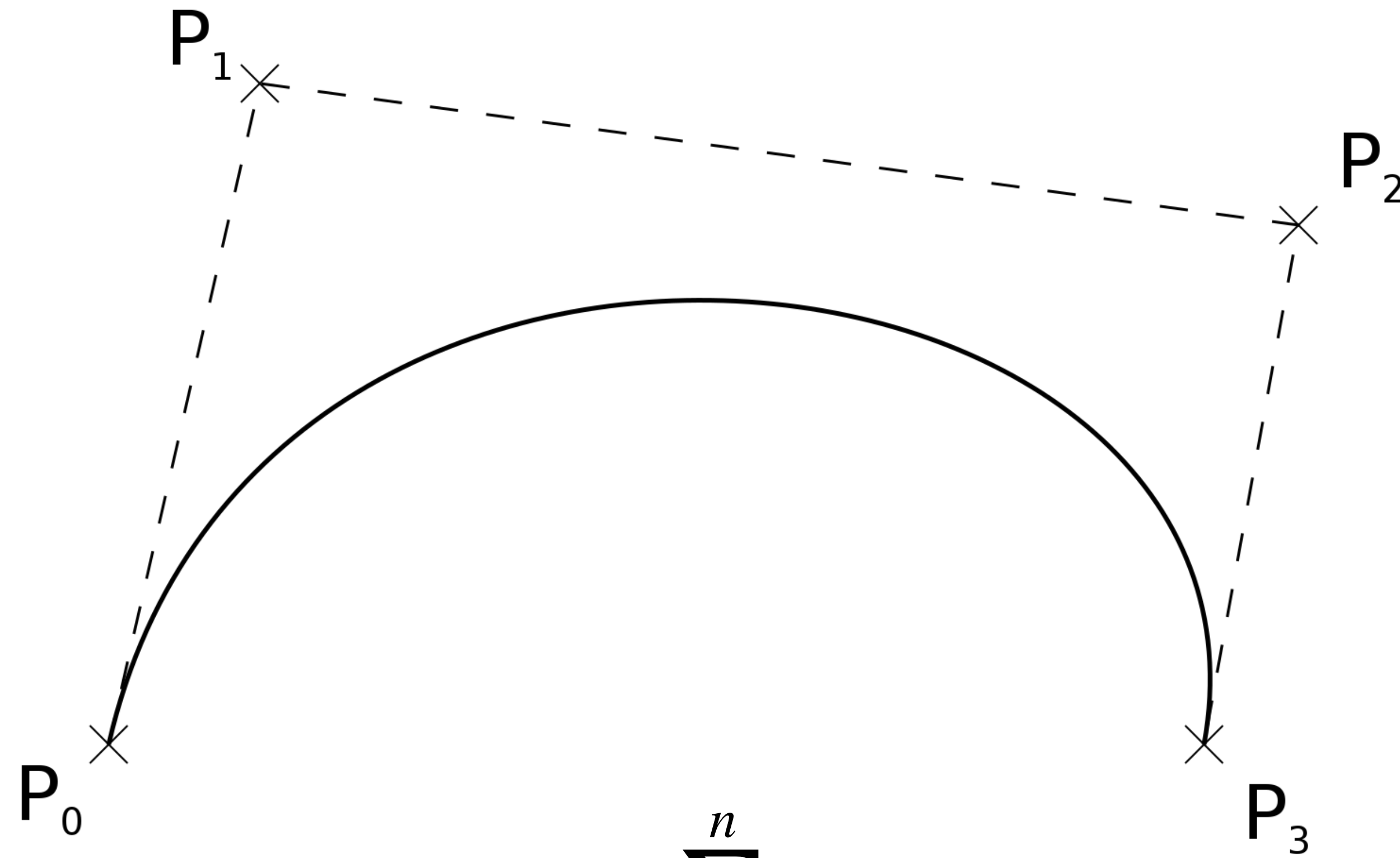
- Free-Form Deformation (Sederberg & Parry, 1986)



- Still widely used today
 - e.g. Blender Lattice modifier



Recall: Bezier Curve from Lecture 1



$$s(t) = \sum_{i=0}^n p_i B_i^n(t)$$

3D Free-Form Deformation

- Control points: 3D lattice
- Modelers drag the vertices of the lattice to define displacements d_i .
- Displacements of points in space are computed by interpolating d_i with interpolating weights B_i

$$d(x) = \sum_i B_i(x) d_i$$

3D Free-Form Deformation

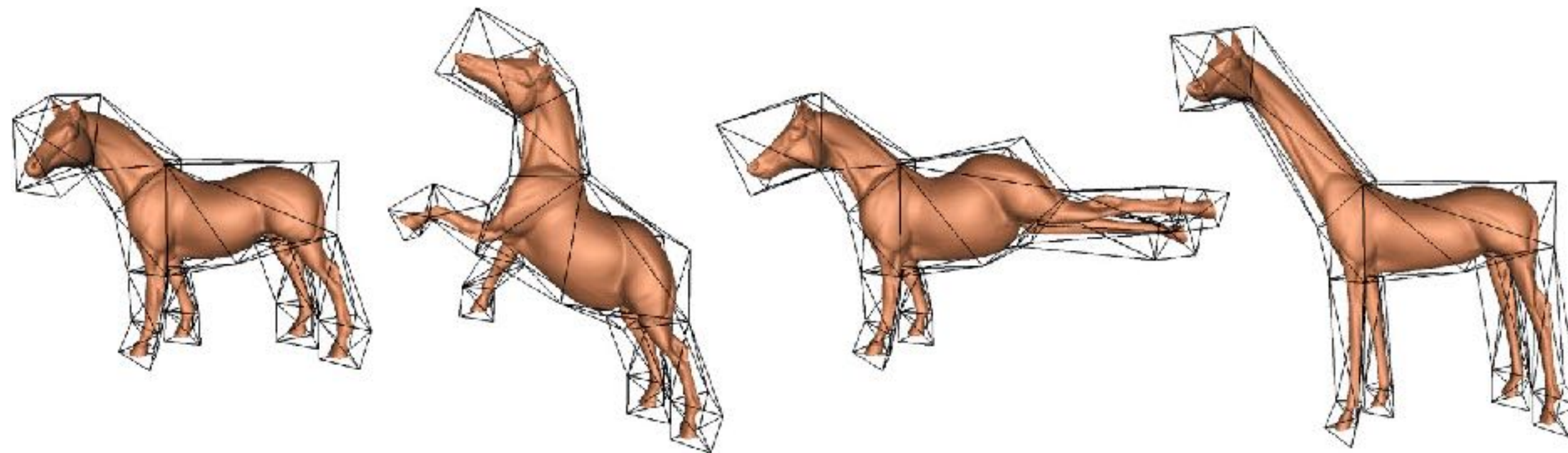
$$d(x) = \sum_i B_i(x) d_i$$

- Compute the Bezier parameters in each dimension and apply **tricubic** interpolation.

$$d(x, y, z) = \sum_i \sum_j \sum_k B_i(x) B_j(y) B_k(z) \mathbf{d}_{ijk}$$

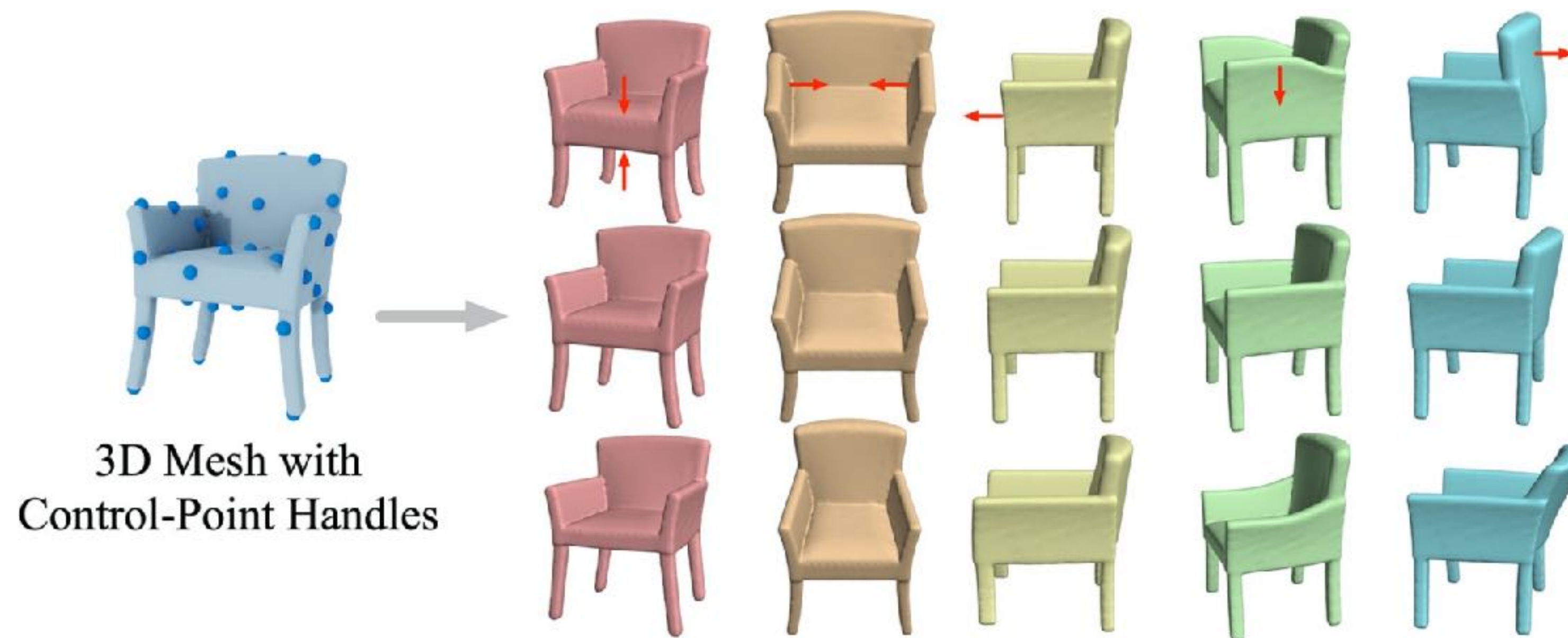
Issues

- Lattice can be large. Modelers do too much: move control points one by one by hand.
- Like Bezier curves, not easy to intuitively relate position of control points with the geometry.
- There are approaches using fewer point points, e.g., cage deformation, key-point based deformation



Learning-based Deformation Field by Keypoints

- Use keypoints as control points
- Use network to learn a basis function from data!



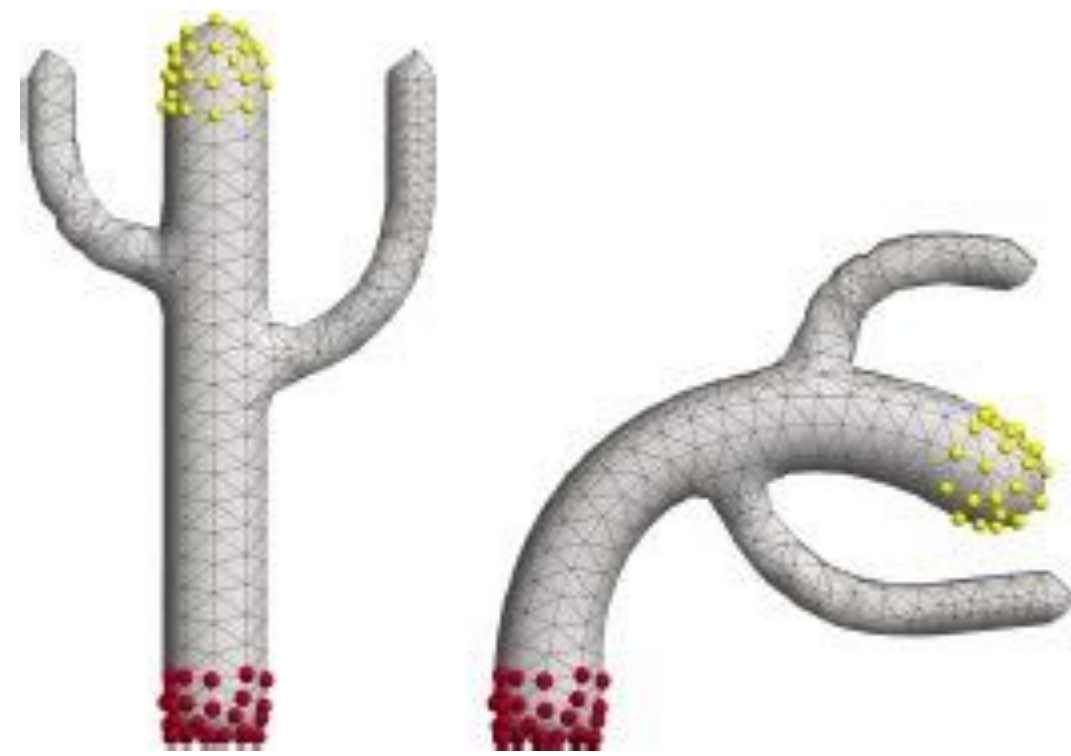
Summary of Space Deformation

- Space deformation are typically very fast
- Run in real time
- Widely used in real-time animation

Outline

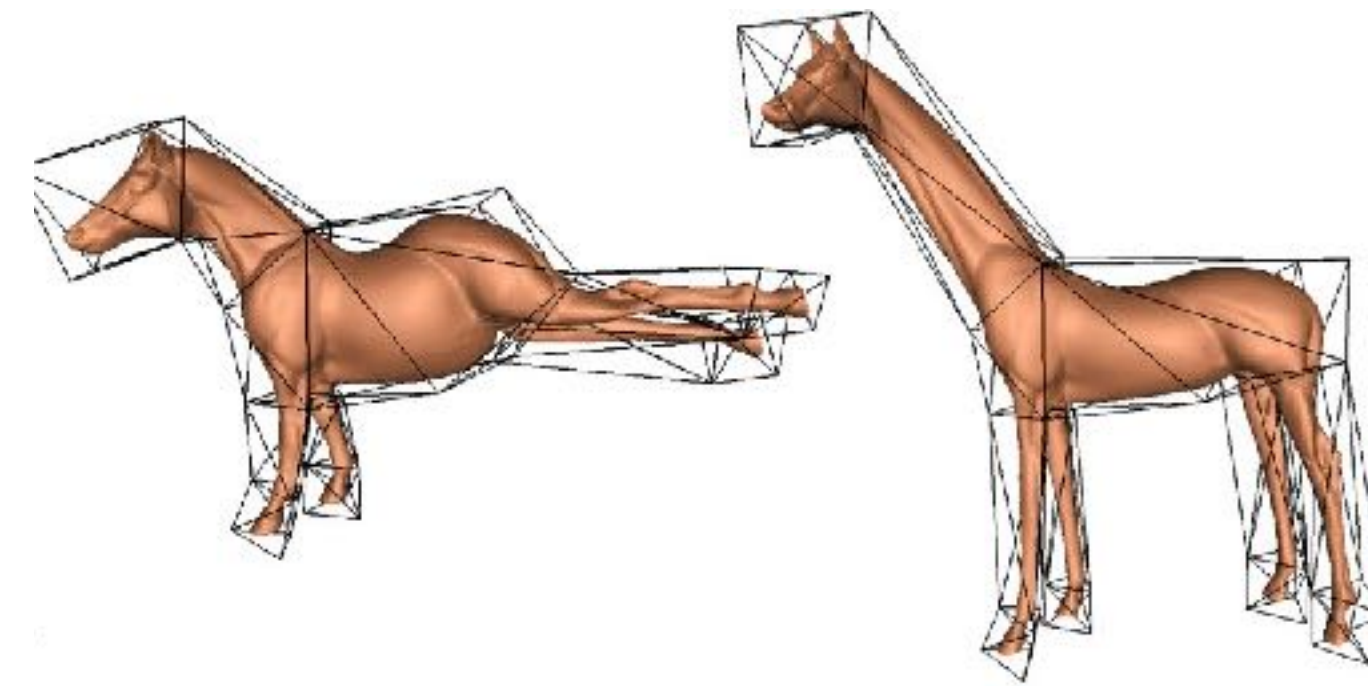
- Introduction
- Surface Deformation
- Space Deformation
- *Skeleton Skinning*

Boneless Shape Editing



Surface Deformation

- Pro: Automatically preserve curvatures
- Con: Slow

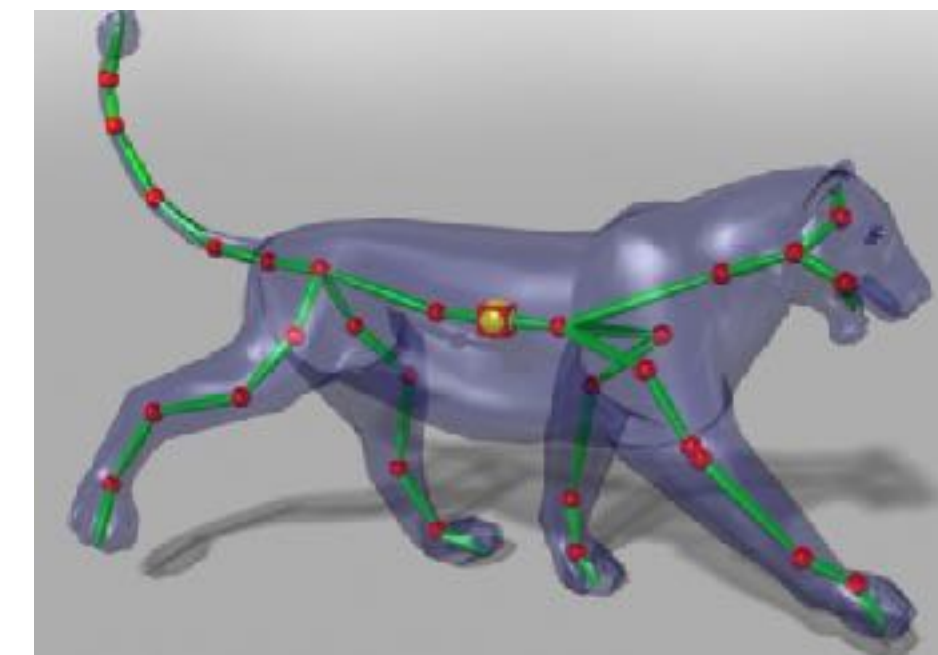
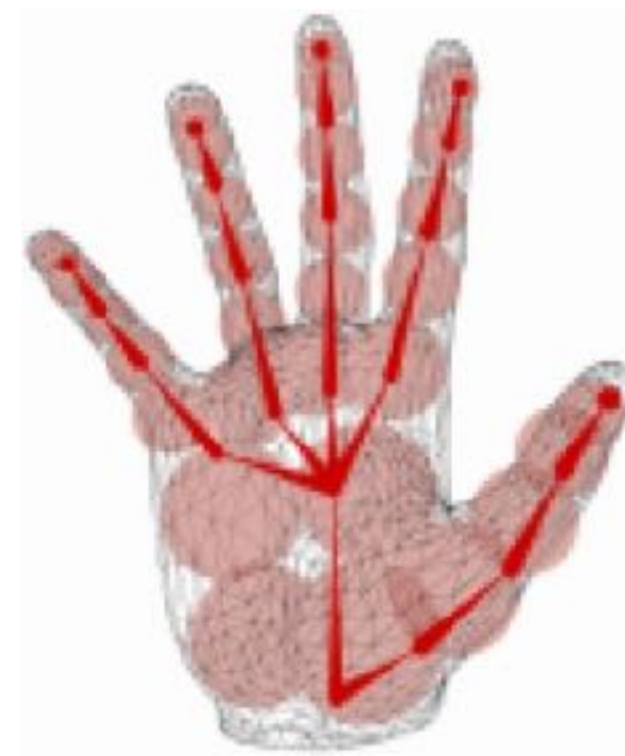
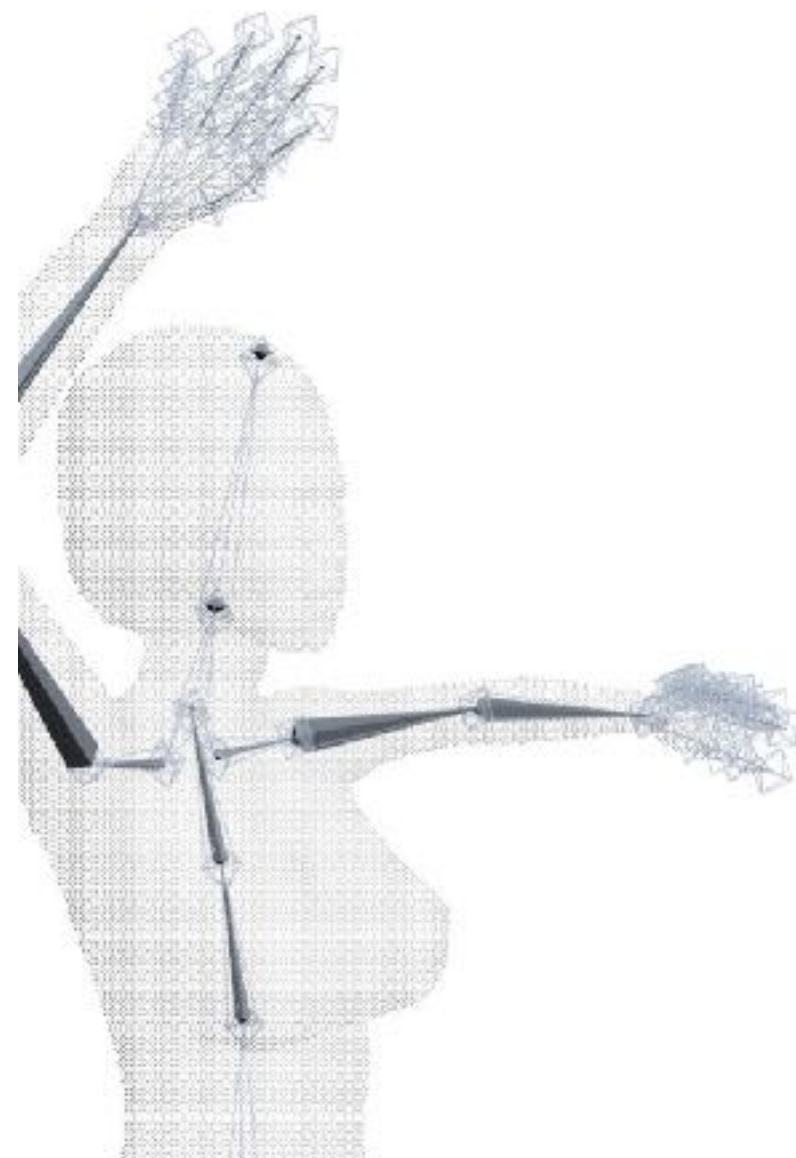


Space Deformation

- Pro: Fast
- Con: Need artists to tune control point movements to achieve curvature preservation

Deformation for Objects with Bones

- Many objects have “bones” – deformation may be interpreted as
 - coarse-level bone transformation; and
 - fine-level skin transformation



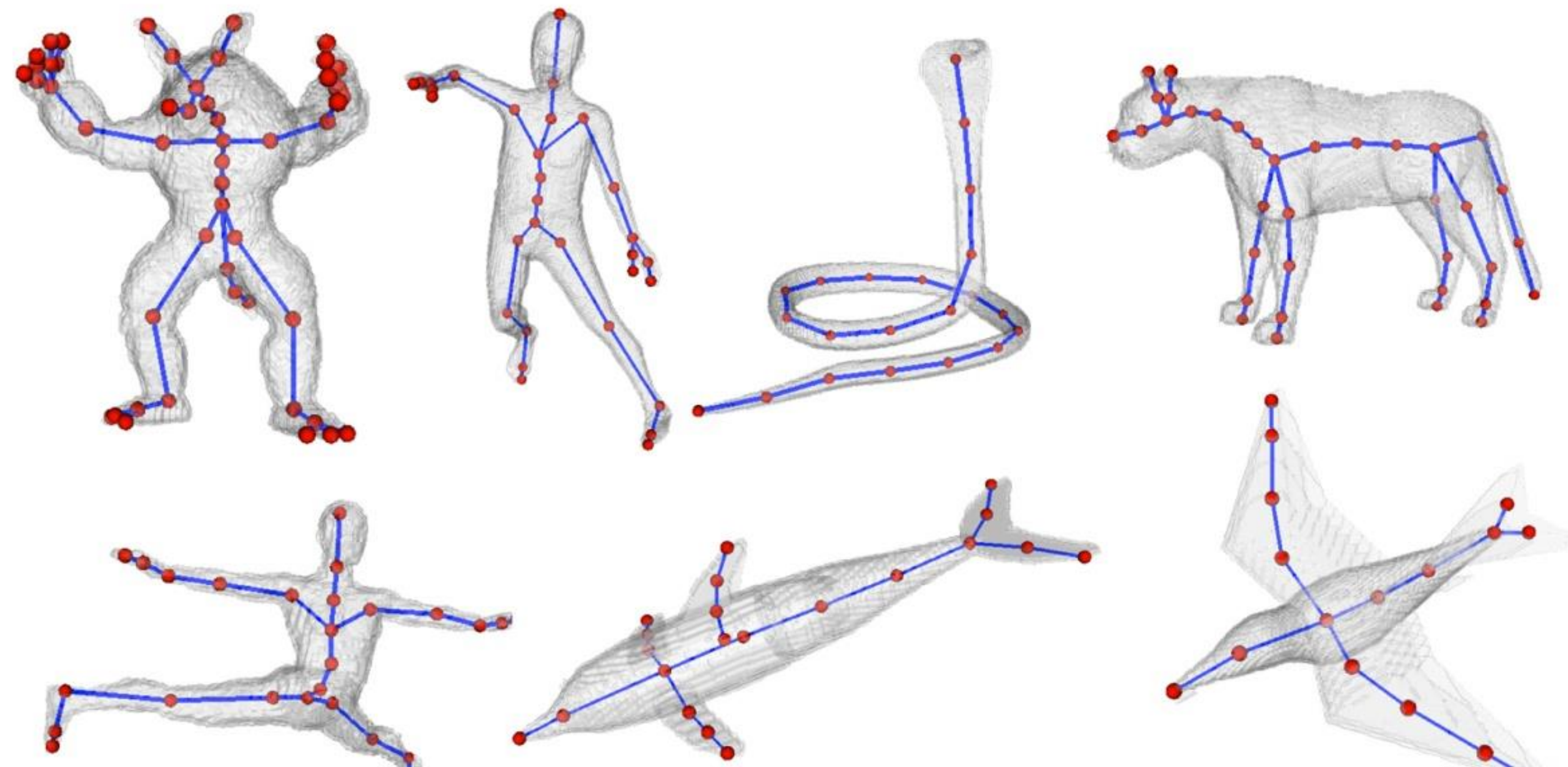
Kavan, et al. Direct Skinning Methods and Deformation Primitives

Romero, et al. Embodied Hands: Modeling and Capturing Hands and Bodies Together

Le, et al. Robust and Accurate Skeletal Rigging from Mesh Sequences

Skeleton

- A skeleton is a framework of rigid body “bones” connected by articulated joints
- Used as an (invisible?) armature to position and orient geometry (usually surface triangles)



Joints

- Joints connect rigid bodies together, while allowing for relative motion between them
- Different types: hinge, ball-and-socket, saddle joint, sliding...
- A Joint has 0-6 degrees of freedom (DoF)
 - A 0-DoF joint rigidly connects two bodies into a single rigid body
 - A full 6-DoF joint doesn't do anything, and each of the bodies are free to move entirely independently
 - Typical joints are somewhere in-between (i.e. from 1-5 DoF)

Joints

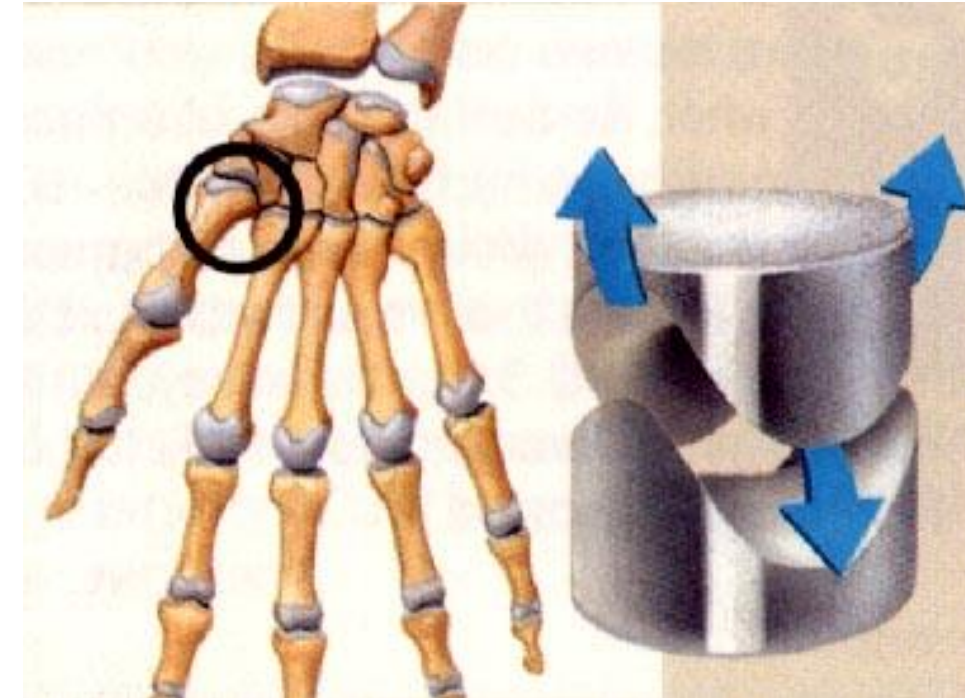
- Rotational Joints
 - 1-DoF: a rotation matrix $\text{Rot}_{4 \times 4}(\mathbf{n}, \theta)$ defined by axis $\mathbf{n}$ and angle θ
 - 2-DoF: multiplication of two sequential rotation matrices about different axes
 - 3-DoF: multiplication of 3 sequential rotation matrices about 3 different axes
 - Like Euler Angles, has the same problem of Gimbal lock
 - Better to specify a 3D rotation about an arbitrary axis (quaternion-style)
- Translational Joint
 - Can be specified to translate along any axis: (1-DoF, 2-DoF, 3-DoF)
 - Translation matrix $T_{4 \times 4}(\mathbf{v})$ is defined by the translation vector $\mathbf{v}$
- Compound Joint
 - Combines rotational and translational joints together

Joints



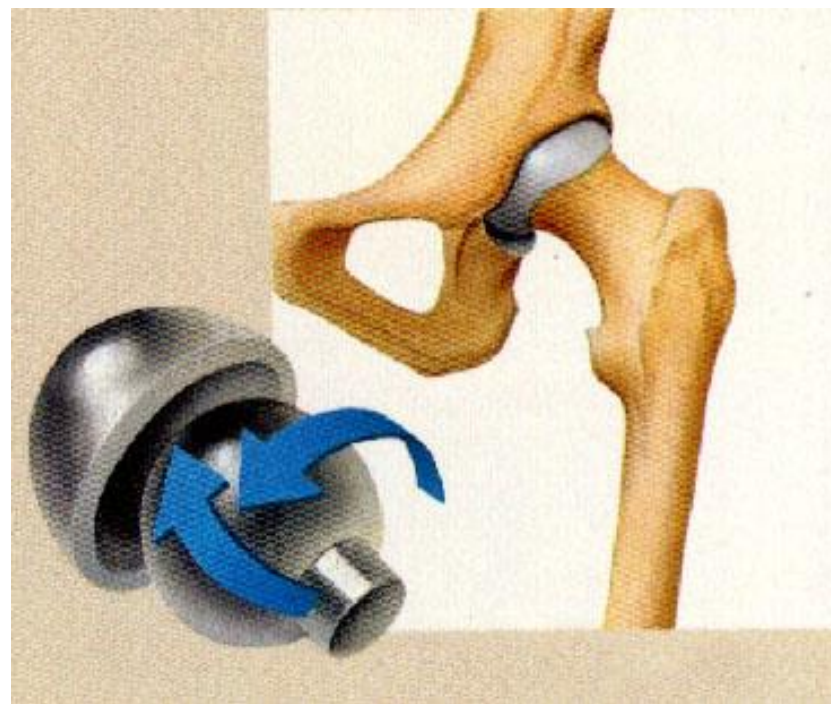
Hinge

- rotation along one axis
- DoF=1



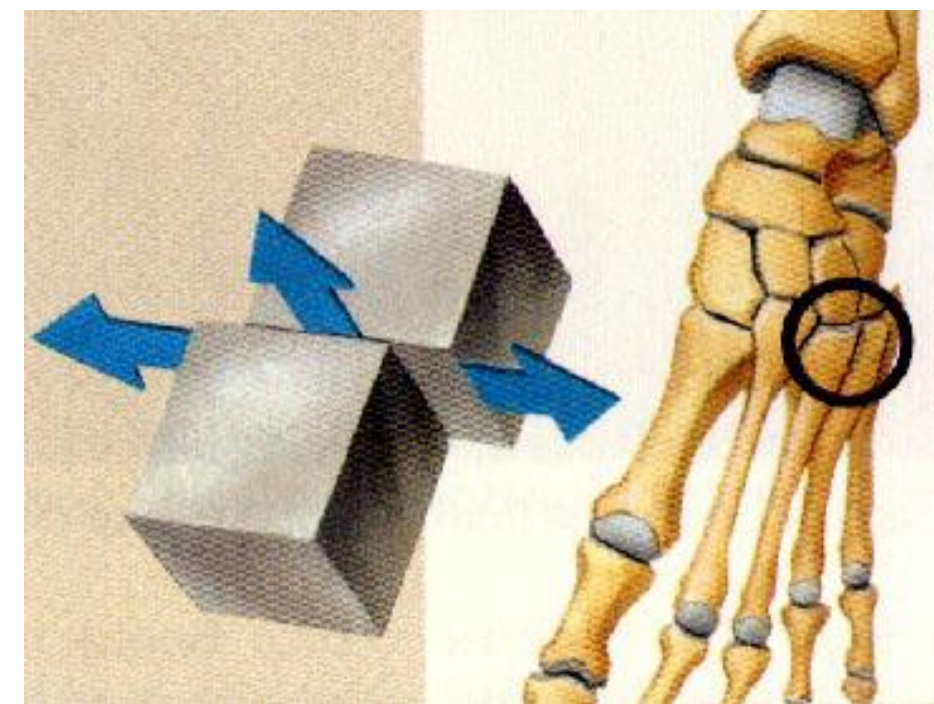
Saddle

- back and forth & up and down motion
- no “rotation”
- DoF=2



Ball-and-socket

- rotation along all 3 axes
- DoF=3



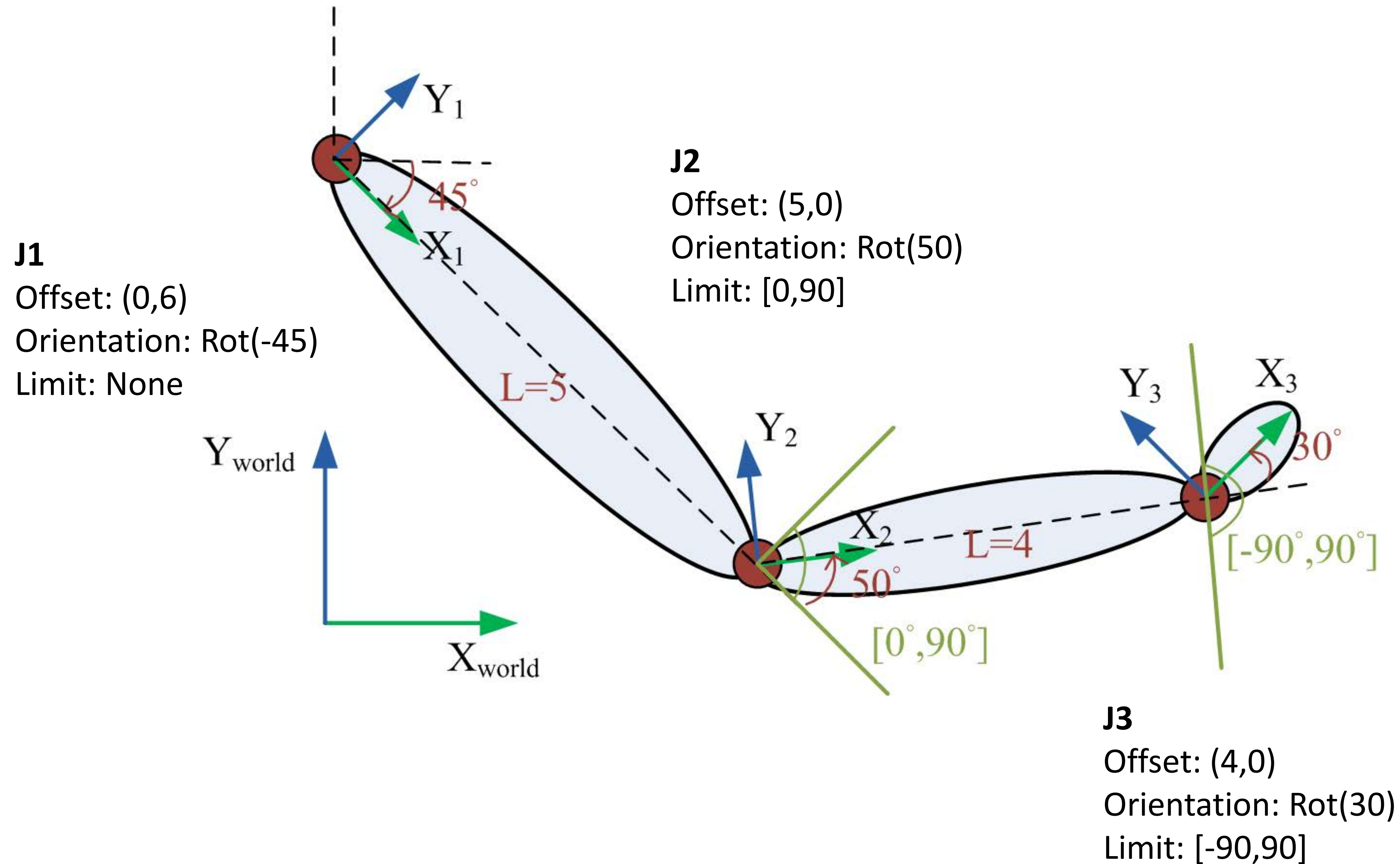
Sliding

- sliding in a plane
- DoF=2

Joint Parameters

- Offset
 - A **fixed** translational displacement in the space of the parent body/joint, which acts as a pivot point for the joint's movement
- Orientation
 - Orientation of a joint's local coordinate system defined in the space of its parent body/joint (a matrix or quaternion)
 - If using homogeneous coordinates, offset and orientation can be defined together in one 4x4 matrix
- Joint Limits
 - The minimum and maximum limits for each DOF that can be enabled or disabled independently
 - E.g. the human elbow can bend to about +150 degrees and hyperextend back as much as -10 degrees

Joint Parameters



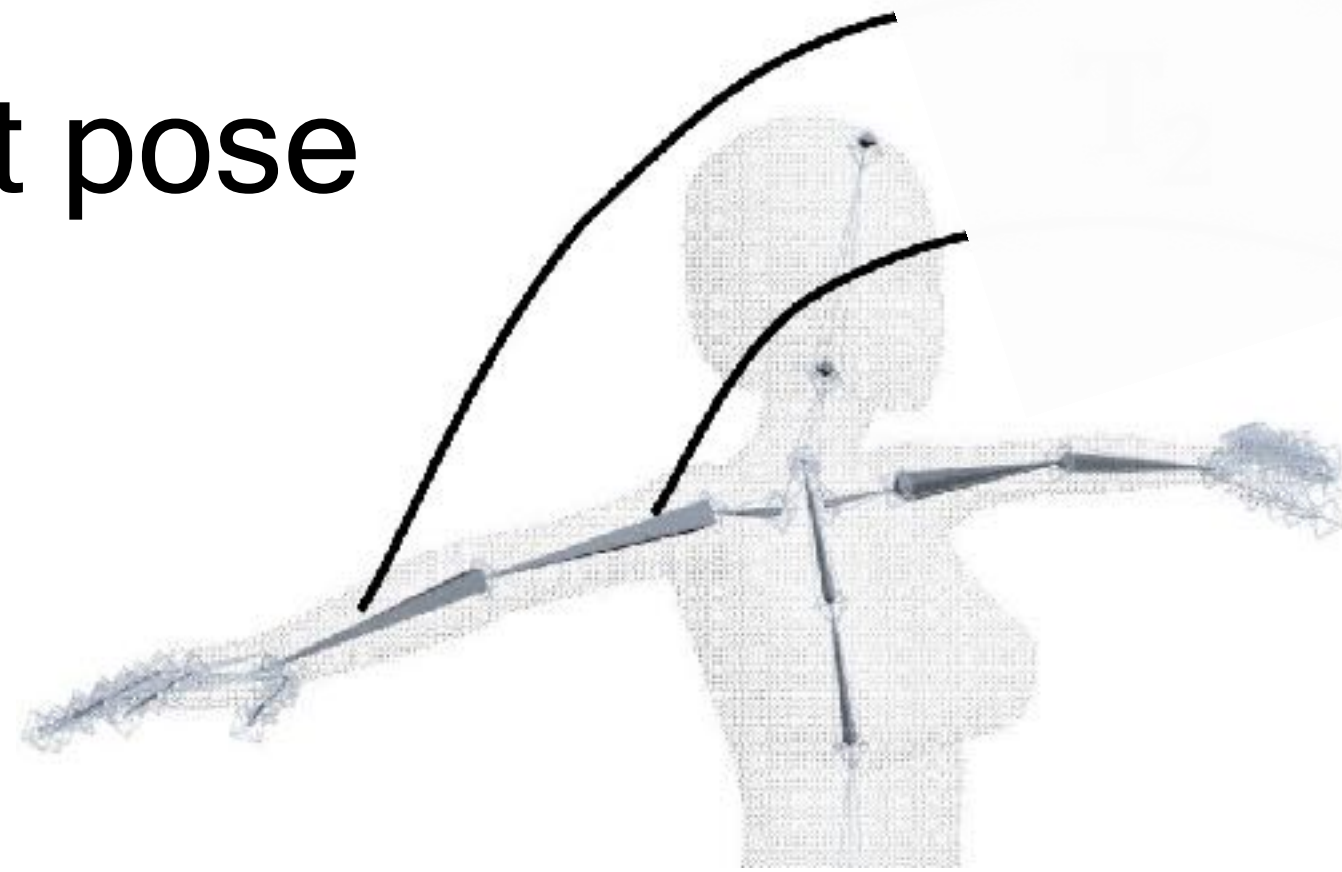
Skeleton

- Skeleton: bones of body linked together
- The pose of bones can be represented using a set of matrices $T_i \in SE(3)$ from rest pose to current pose

Skeleton

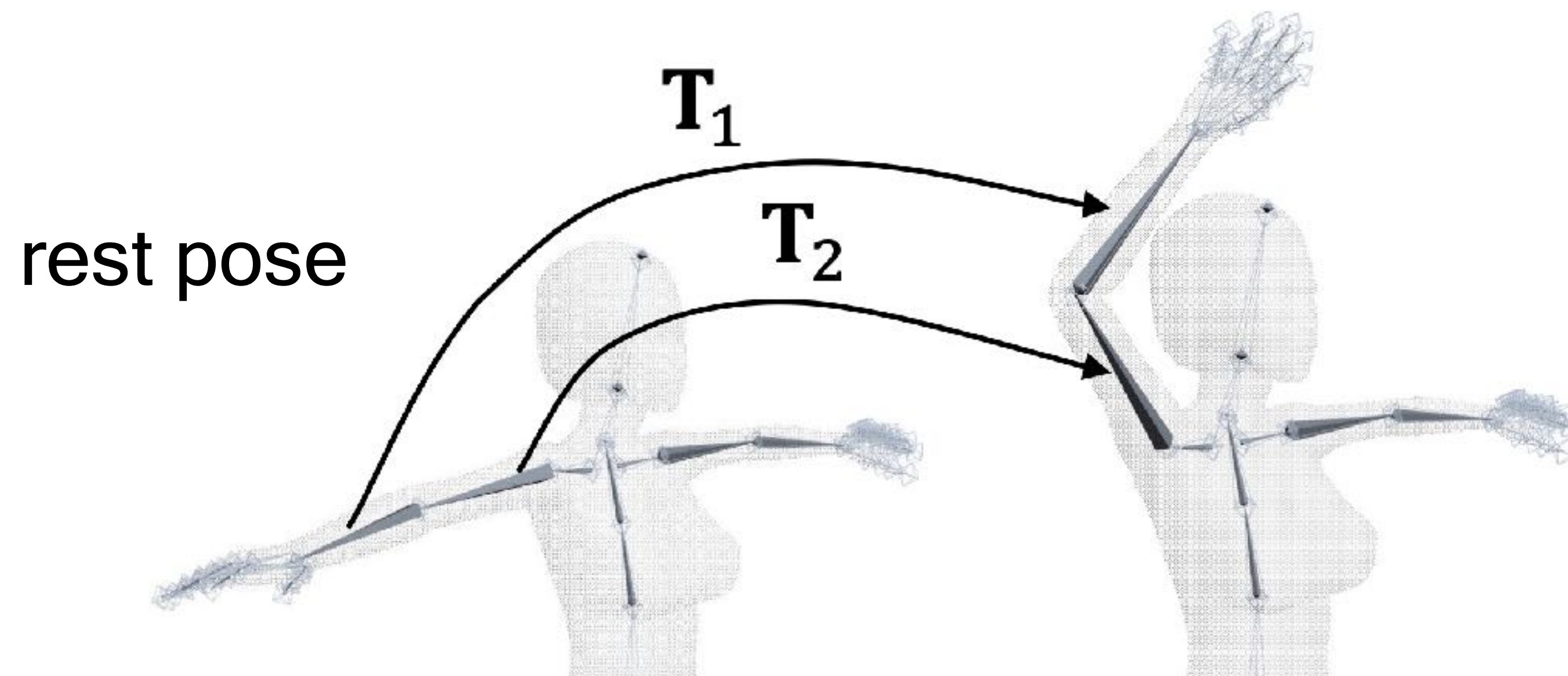
- Skeleton: bones of body linked together
- The pose of bones can be represented using a set of matrices $T_i \in SE(3)$ from rest pose to current pose

rest pose



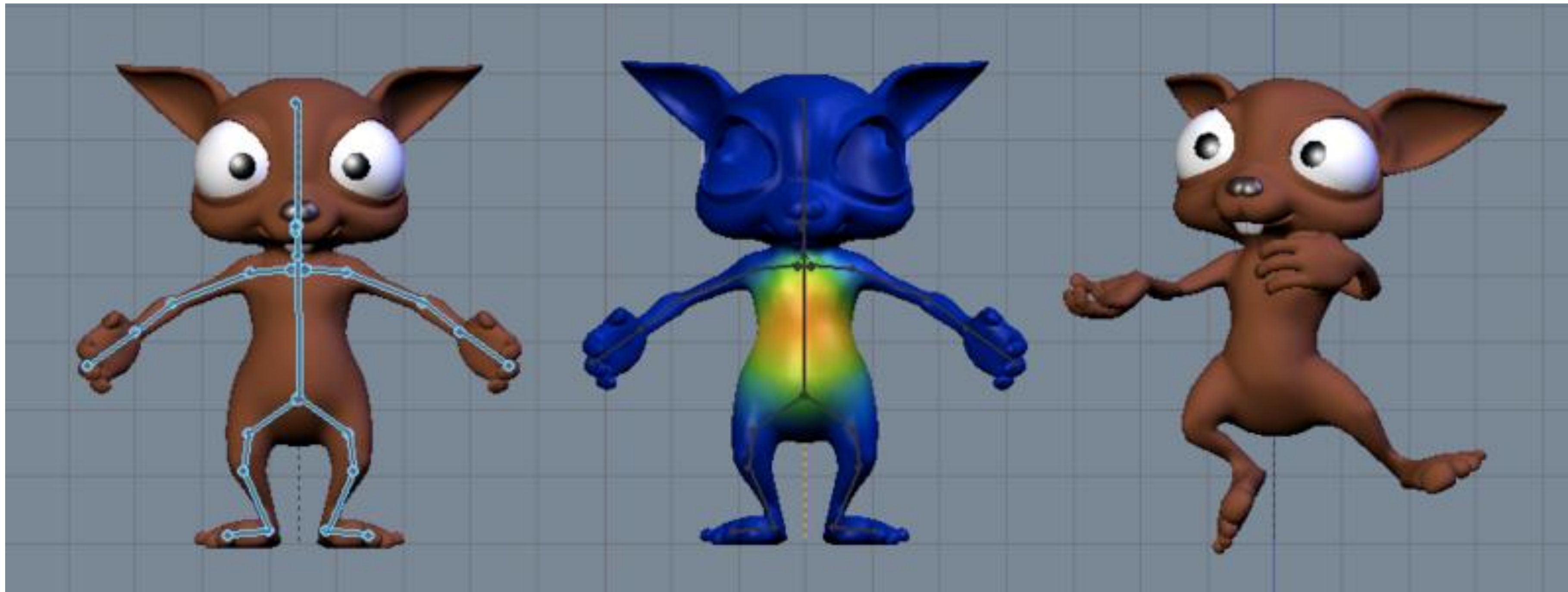
Skeleton

- Skeleton: bones of body linked together
- The pose of bones can be represented using a set of matrices $T_i \in SE(3)$ from rest pose to current pose



Skinning (or Enveloping)

- Envelop the underlying skeleton with a surface representation (triangle mesh, implicit surface), or skin, that conveys the appearance of the character and deforms with the underlying skeleton



Joint Angles Framework

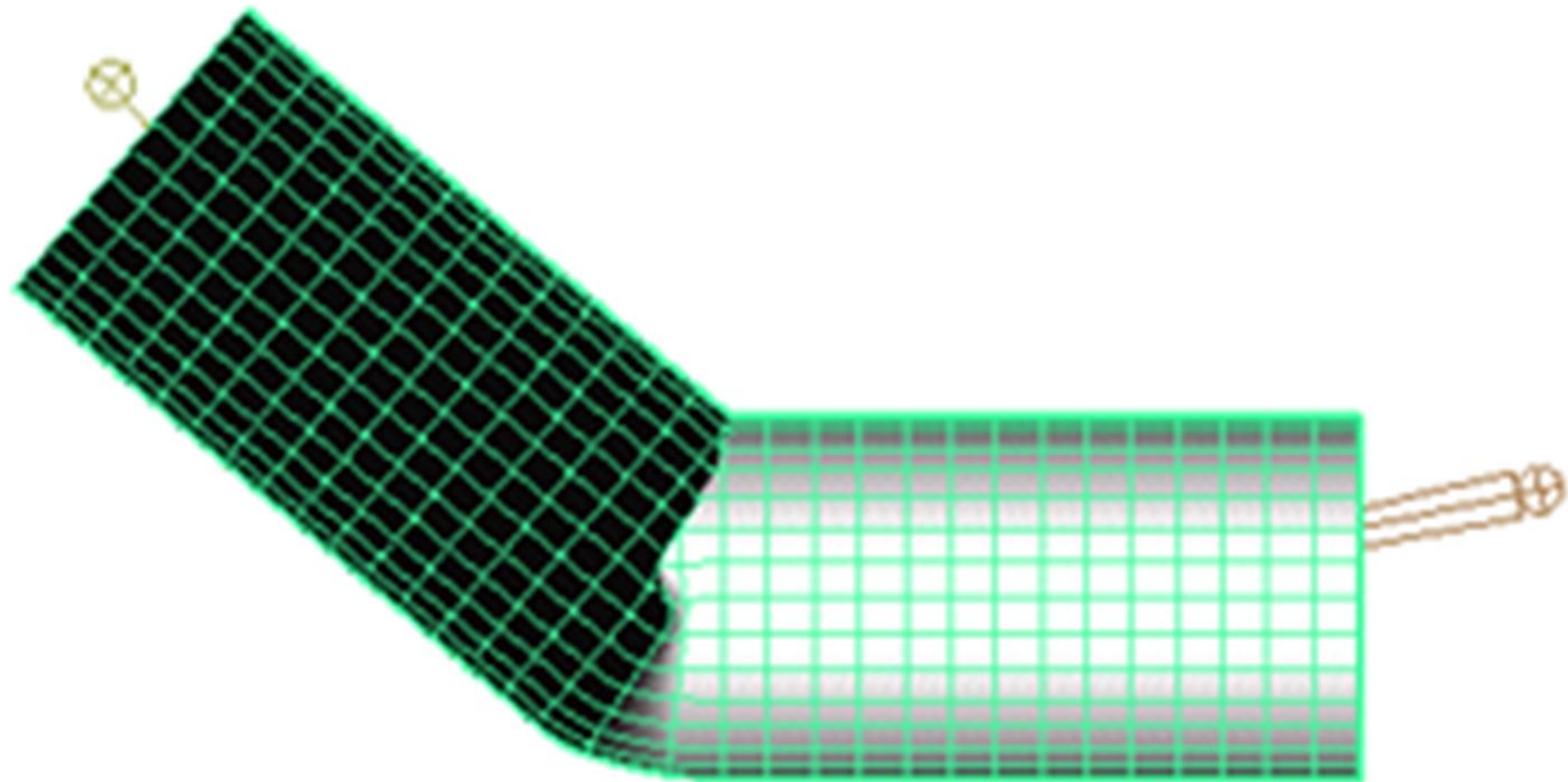
- Decompose the entire skin (for the whole character) into smaller pieces, and place a portion of the skin into the object space of each bone
 - The pieces may overlap, i.e. multiple bones may share the same skin vertices
- Given a set of joint parameters θ
- Let $T_i(\theta)$ represent the transformation that moves bone i from its object space to world space
- As the joint parameters change and the bones move in world space, calculate where the skin vertices are located in world space as well using $T_i(\theta)$
- Skin vertices which exist in the object space of multiple bones require some sort of interpolation or averaging

Rigid Skinning

- Each skin vertex is assigned to exactly one bone
 - For example, the skin for the upper arm would be assigned to a different bone than the skin for the forearm
- Use the transform of the associated bone to position each vertex of the skin in world space:
 - Consider a vertex j with position v_j in the object space of the i th bone with transformation T_i
 - Then, the world space position of vertex j is given by
$$v'_j = T_i v_j$$
- As the skeleton moves, T_i changes and the vertex positions of the skin change as well

Rigid Skinning

- Unwanted discontinuities form along the boundaries where neighboring skin vertices are assigned to different bones



Linear Blend Skinning

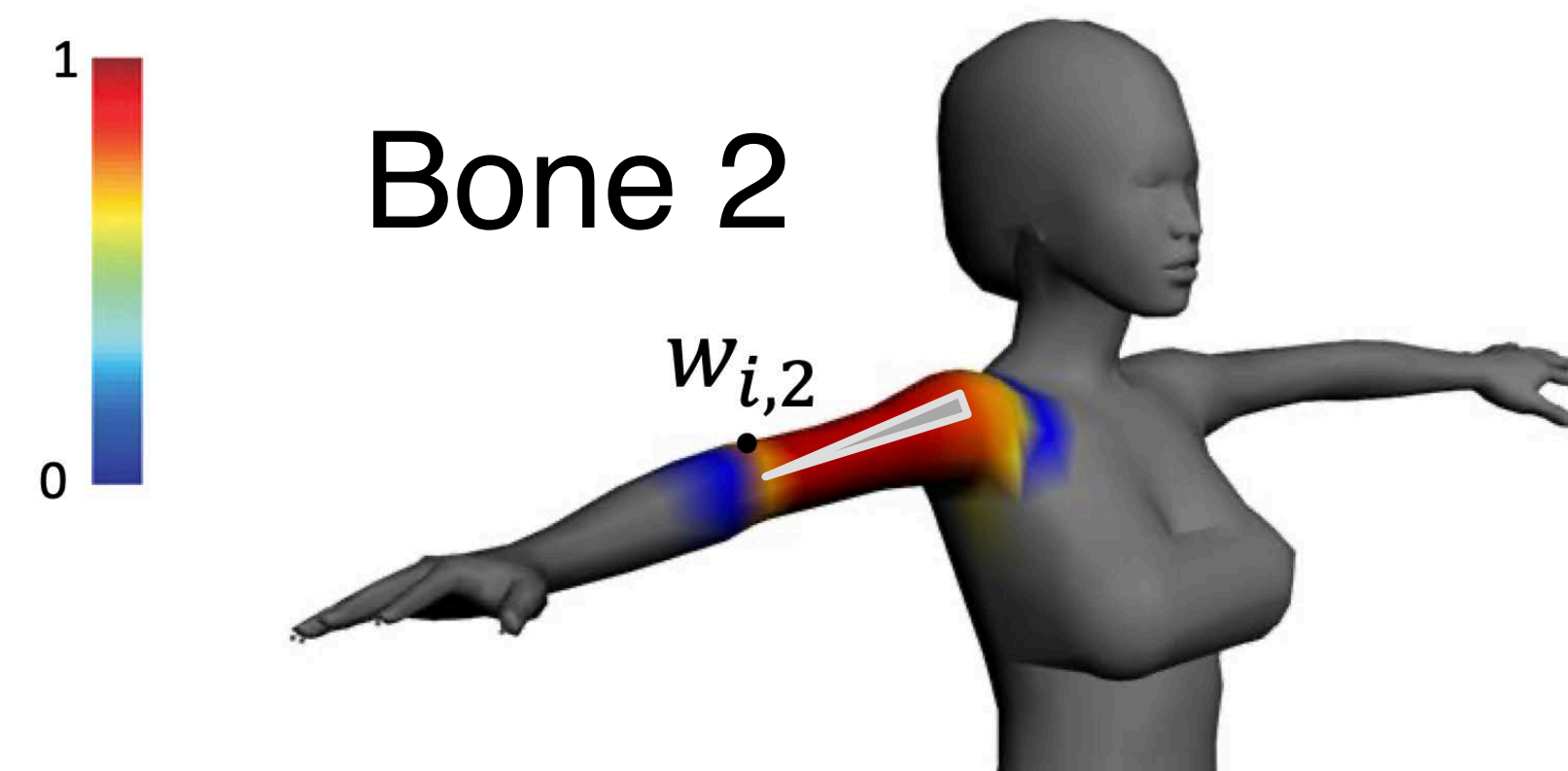
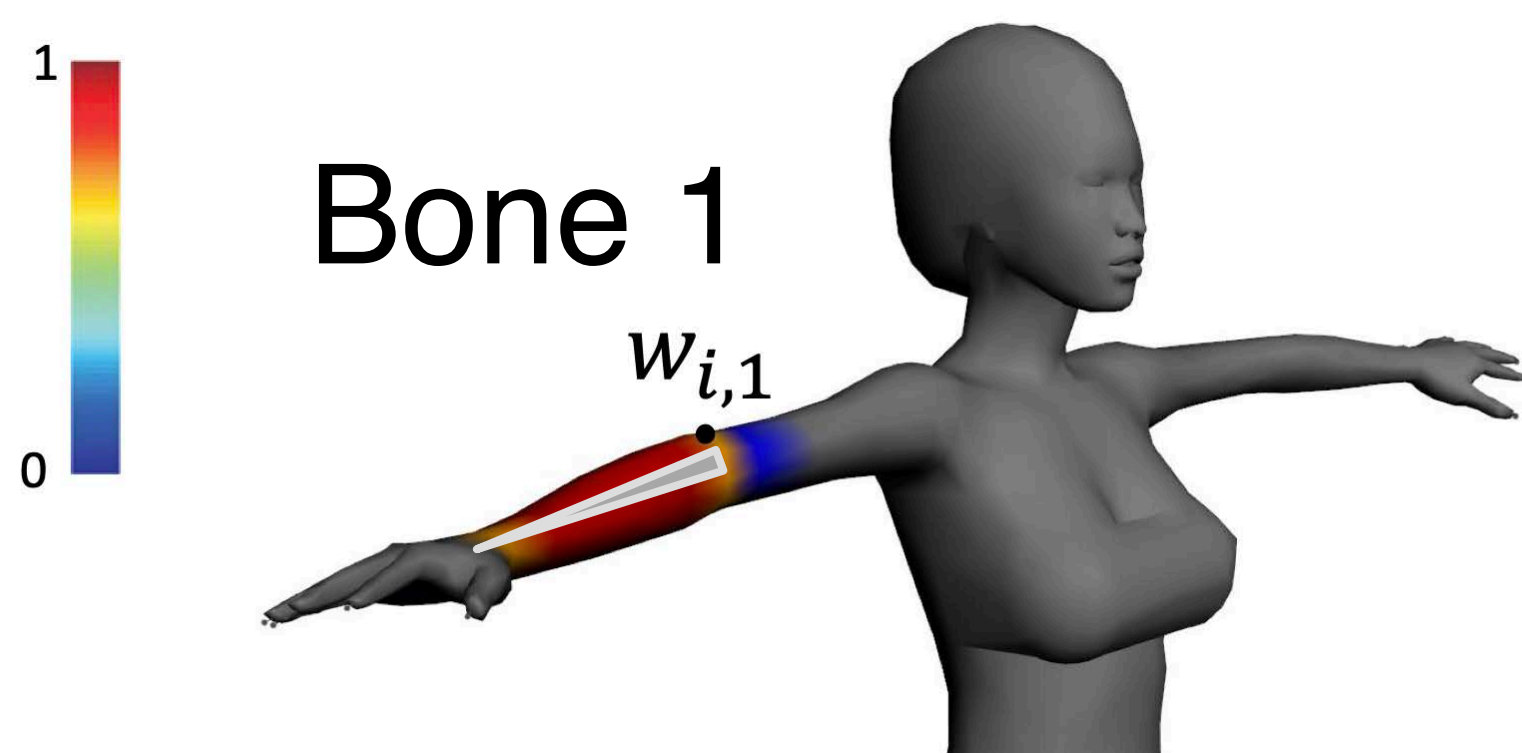
- Skin vertex move when pose of bone T_j change
 - If v_i on the j -th bone, then it will move to $T_j v_i$
- Around joints there will be cracks. In practice, each vertex is governed by multiple bones,
 - e.g., averaged by a **linear** model (SMPL):

$$v'_i = \sum_{j=1}^m w_{i,j} T_j v_i = \left(\sum_{j=1}^m w_{i,j} T_j \right) v_i$$

- $w_{i,j}$: skinning weights
 - Describes the amount of influence of bone j on vertex i

Skinning Weights

- We commonly require $w_{i,j} \geq 0$, $\sum_{j=0}^m w_{i,j} = 1$



Skinning Weights

- Weights for a vertex should be sparse
 - E.g., if the angle of the elbow joint is changed, the skin for the leg shouldn't deform
 - Nonzero weights should be localized to nearby bones
- Sparse weights allow for fast evaluation
 - Typically at most four non-zero weights per vertex (at most four bones can deform a vertex)
- Weights should be smooth to avoid discontinuities
 - Often chosen with a smooth falloff based on distance to a particular bone
- Weights should be independent of mesh resolution
 - So that subdividing the mesh doesn't require recomputing weights
- Constrain the weights to be convex (i.e. $\sum_i w_{ij} = 1, w_{ij} \geq 0$) to avoid undesired scaling and extrapolation artifacts

Specifying Weights

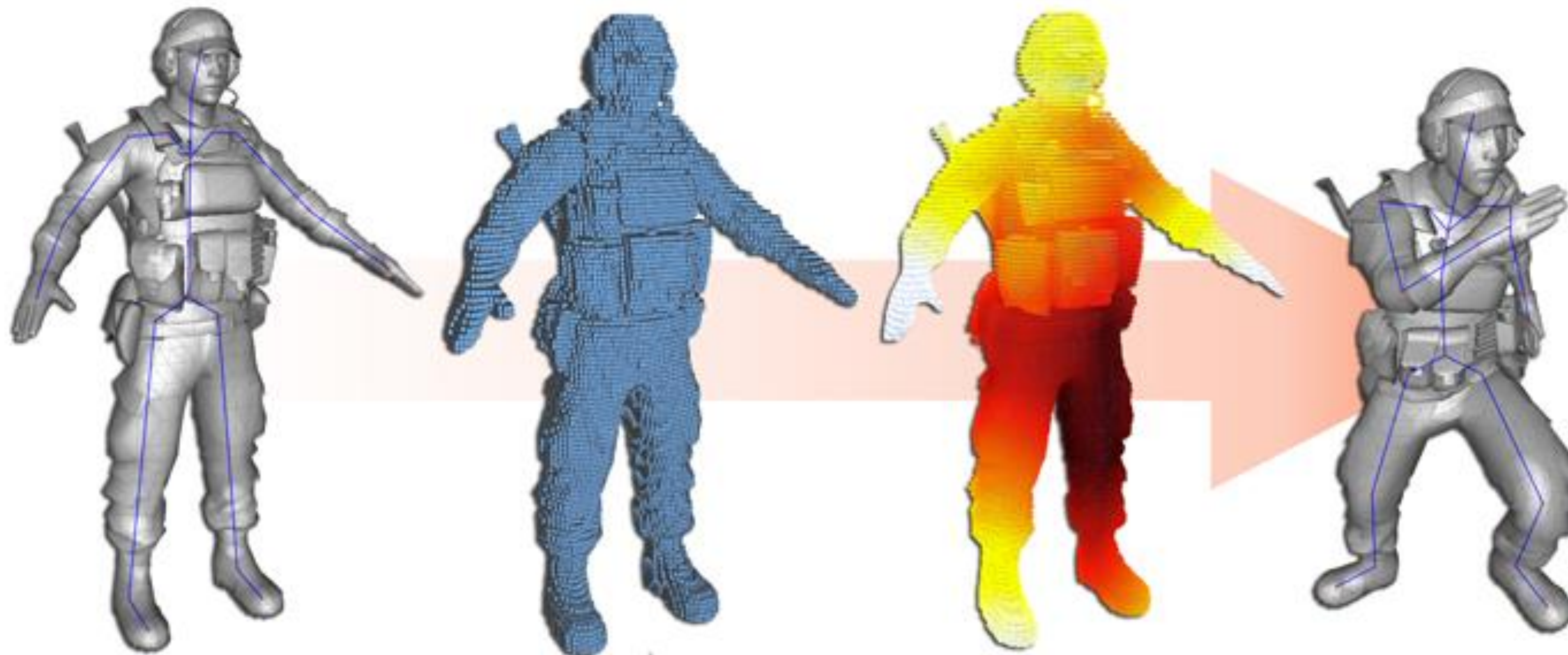
- Manual Approach:
 - Hand-tune weights in order to obtain the best look
 - Intractable to individually modify the weights for each vertex in a large mesh
 - Various painting tools facilitate weight specification
- Automatic Approach:
 - Use an algorithm to calculate weights for each vertex and all its associated bones
 - E.g., based on a “distance” metric from vertices to bones
 - Automatically generated weights are often additionally modified by an artist for higher visual fidelity

Specifying Weights: Pinocchio

- System for automatically rigging and animating 3D characters
- Solves a Poisson equation (PDE!) for each bone with appropriate boundary conditions to obtain smoothly varying weights
- Can be used to rig and skin your own characters
- Available from MIT:
<http://www.mit.edu/~ibaran/autorig/pinocchio.html>

Specifying Weights: Geodesic Voxel Blending

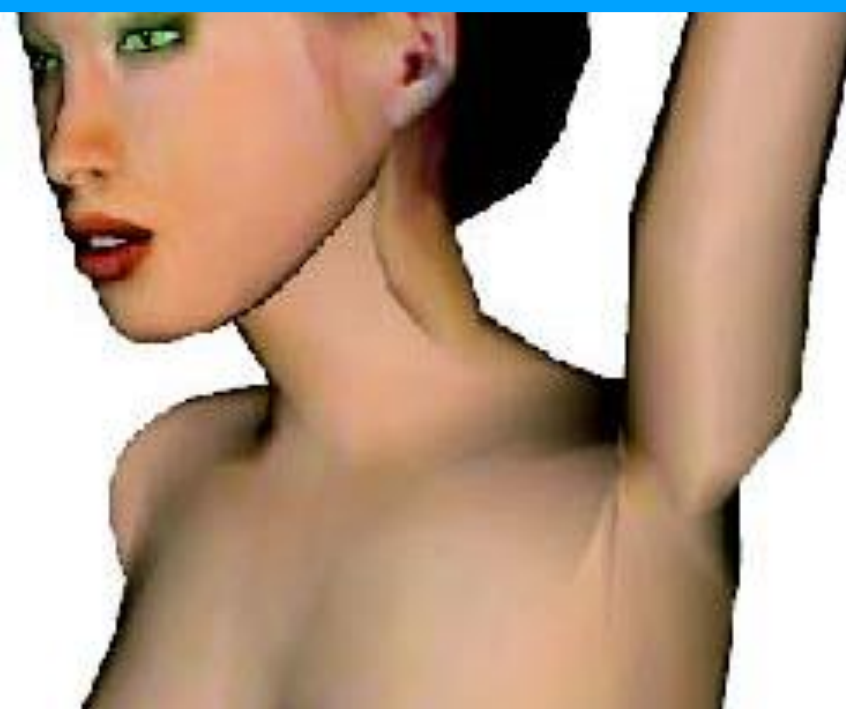
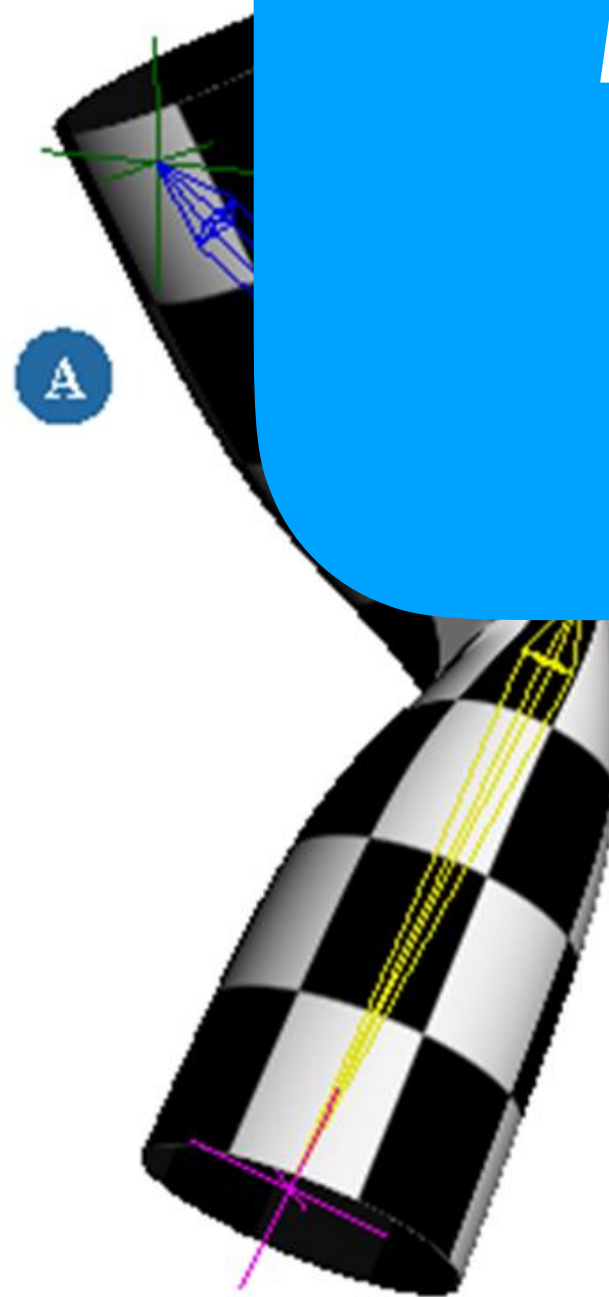
- Automatic approach for specifying weights:
 1. Voxelize the interior and boundary of the skin mesh for a rest pose
 2. For each bone (left leg bone shown below), compute the geodesic distance from that bone to the center of each voxel using the Fast Marching Method
 3. For each skin vertex, interpolate distance from the surrounding voxels
 4. Use this distance in a falloff function to determine the weight for each vertex



Artifacts

- Linear blend skinning has issues when the joint angles are large or when a bone undergoes a twisting motion
 - “bow t”
 - mesh
- Linearly transform represent

How to address this issue?
Dual Quaternion Skinning



Dual Numbers

- A dual number has the form $\hat{a} = a_0 + \epsilon a_\epsilon$
 - where a_0 and a_ϵ are real numbers
 - and ϵ satisfies $\epsilon^2 = 0$
- Many arithmetic operations are defined for dual numbers
 - such as multiplication, division, conjugation, and the square root
- Dual numbers can be represented as 2x2 matrices:
$$\epsilon = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \text{ and } aI + b\epsilon = \begin{pmatrix} a & b \\ 0 & a \end{pmatrix}$$
- The sum and product of dual numbers can then be calculated with matrix addition and multiplication

Dual Quaternions

- A dual quaternion has the form $\hat{q} = q_0 + \epsilon q_\epsilon$
 - where q_0 and q_ϵ are standard quaternions
- If $q_\epsilon = 0$, the dual quaternion reduces to a standard quaternion q_0 (and represents a rotation)
- Dual quaternions can be used to represent a translation $\vec{t} = (t_x, t_y, t_z)$ as

$$\hat{t} = 1 + \frac{\epsilon}{2} (t_x \mathbf{i} + t_y \mathbf{j} + t_z \mathbf{k})$$

- A rigid body transformation with rotation q_0 and translation $\hat{t}$, is represented by the dual quaternion

$$\hat{t}q_0 = \left(1 + \frac{\epsilon}{2} (t_x \mathbf{i} + t_y \mathbf{j} + t_z \mathbf{k}) \right) q_0$$

Skin Space

- Consider a skin space, where the full character skin is placed in its rest pose
- Each bone is placed into skin space and aligned with the character by a rigid transformation B_i
- A vertex v_j in skin space can be placed into the object space of a rigid body via

$$v_j^i = B_i^{-1} v_j$$

- Thus, the full formula for linear blend skinning is

$$v_j' = \sum_i w_{ij} T_i B_i^{-1} v_j$$

- The fact that $\sum_i w_{ij} T_i B_i^{-1}$ is not a rigid body transform leads to some of the issues with linear blend skinning

Dual Quaternion Skinning

- Convert each composite transformation matrix $T_i B_i^{-1}$ into a unit dual quaternion $\hat{q}_i$
- Then compute a normalized linearly blended dual quaternion $\hat{q}_j$ using the weights

$$\hat{q}_j = \frac{\sum_i w_{ij} \hat{q}_i}{\|\sum_i w_{ij} \hat{q}_i\|}$$

- This blended unit dual quaternion is guaranteed to represent a rigid body transformation
- Transform $\hat{q}_j$ back into a transformation matrix $\hat{T}_j$ and calculate the deformed skin vertex position as $v'_j = \hat{T}_j v_j$



Classic linear



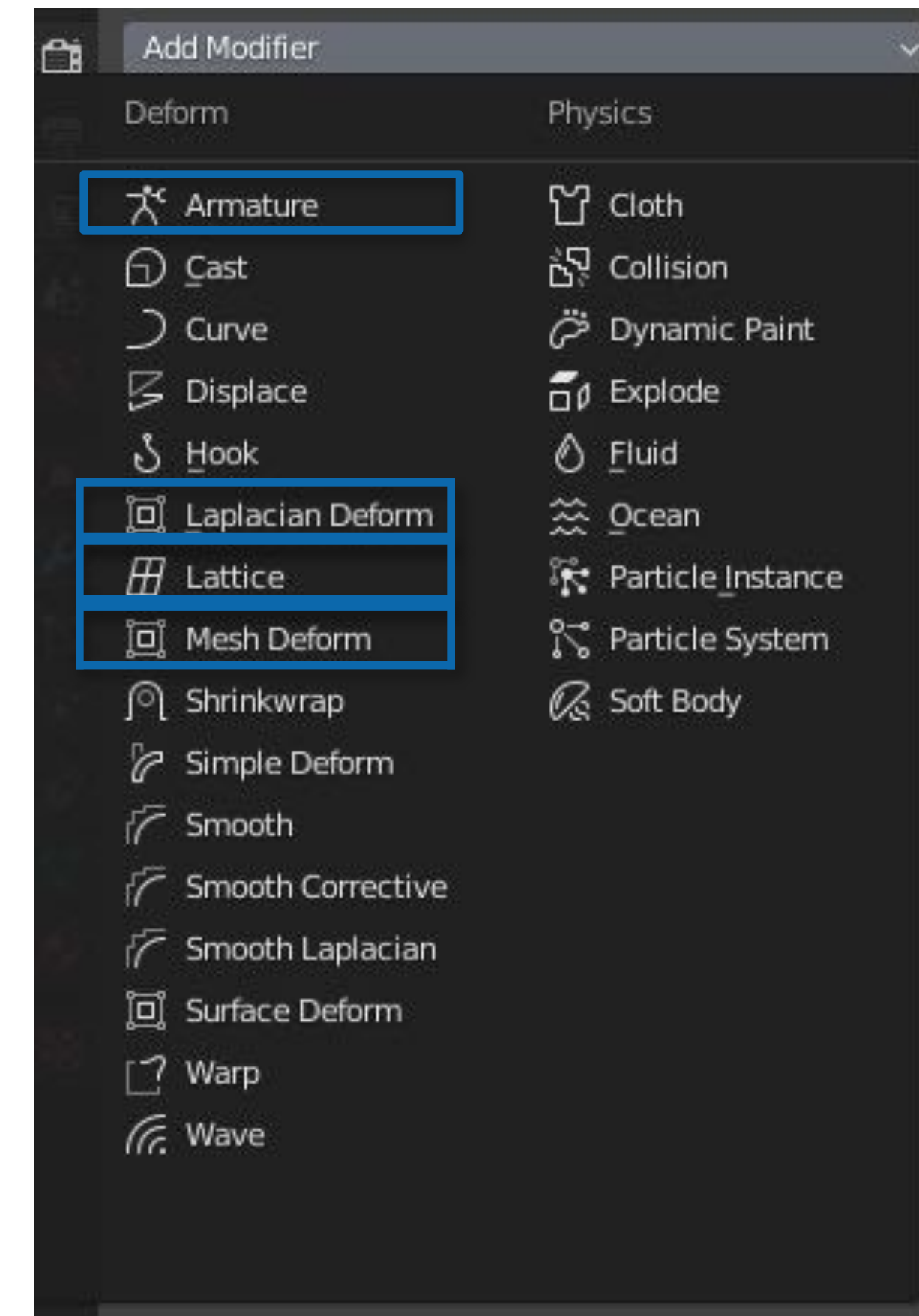
Dual quaternion

Summary of Skeleton Skinning

- Skeleton: linked bones
- Skinning: deform the surface along skeleton transformation
- Linear Blend Skinning:
 - Rest pose
 - Bone transformation
 - Skinning weight

Deformation in Blender

- You can find these deformation algorithms in blender. Try it yourself!
- There are a lot more to play with!



Syllabus

Syllabus for 3D Visual Computing Spring 2025

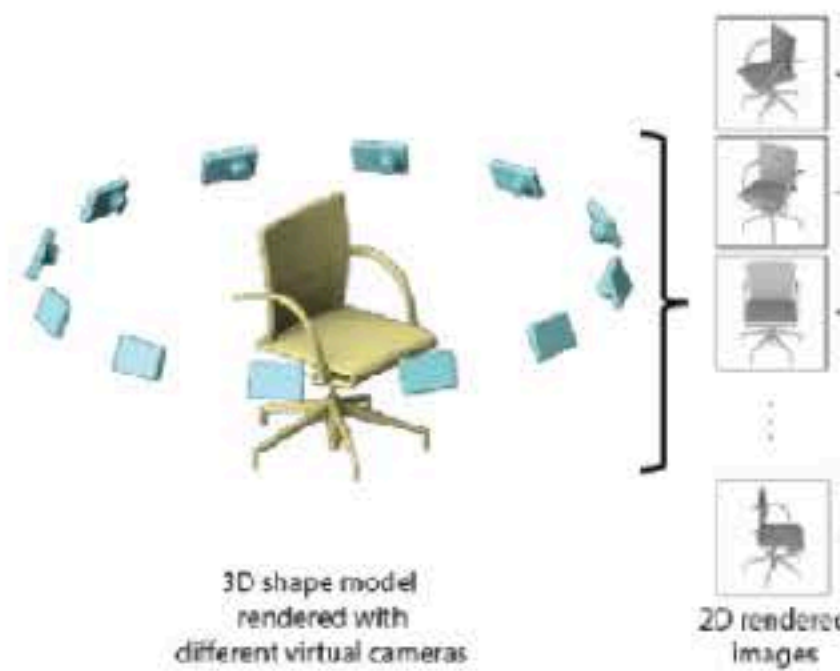
	Geometric Foundation			Apr 17 (Week 9)	Surface Reconstruction	Project Announced
Feb 20 (Week 1)	Curve and Surface	Hw1 Out			3D Analysis	
Feb 27 (Week 2)	3D Representations			Apr 24 (Week 10)	3D Backbone Networks	Hw2 Due;
Mar 6 (Week 3)	3D Transformations			May 1 (Week 11)	Holiday	
	3D Reconstruction and Synthesis			May 8 (Week 12)	3D Backbone Networks II	In-Class Midterm
Mar 13 (Week 4)	Learning-Based SFM and SLAM	Hw1 Due; Hw 2 Out		May 15 (Week 13)	3D Detection and Segmentation	
Mar 20 (Week 5)	Learning-Based Multi-view Stereo			May 22 (Week 14)	Pose Estimation	
Mar 27 (Week 6)	NeRF			May 29 (Week 15)	Intrinsic Shape Understanding	
Apr 3 (Week 7)	Single Image to 3D			Jun 5 (Week 16)	Shape Deformation	
Apr 10 (Week 8)	3D Generative Models			Jun 12 (Week 17)		Project Due

Curve and Surface

- We can use parametrized form to represent smooth curves and surfaces
- Such parametrized form allows us to define local properties such as tangent, normal and curvature
- We usually require “smoothness” for easier analysis.

3D Representations

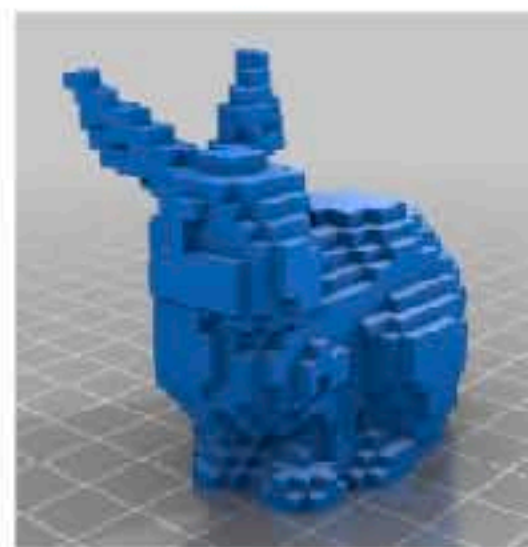
Rasterized form (regular grids)



Multi-view

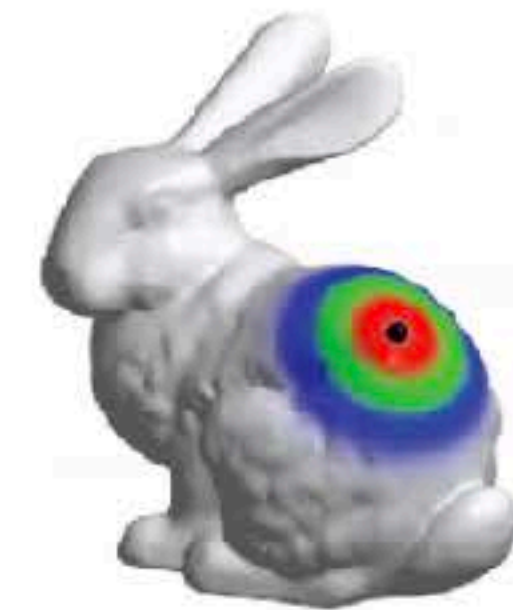


Depth Map

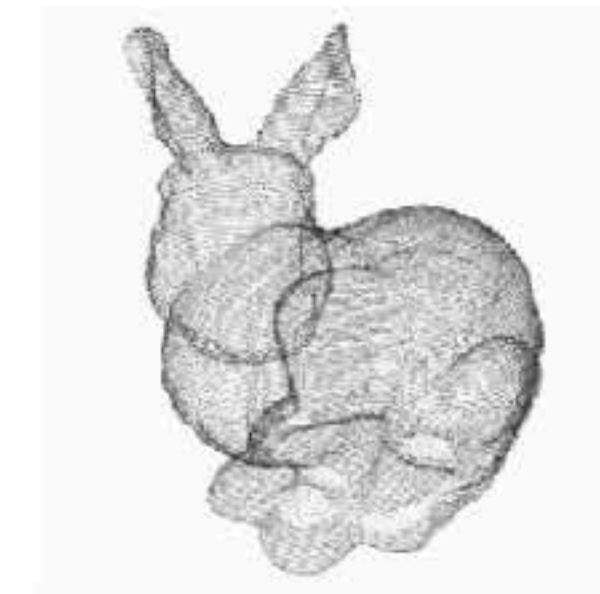


Volumetric

Geometric form (irregular)



Mesh



Point Cloud

$$F(x) = 0$$

Implicit Shape

3D Transformations

- Transformations and homogeneous coordinates
- Rotation and $SO(n)$
- 3D Rotation representation
 - Euler Angles
 - Axis-Angle
 - Quaternion

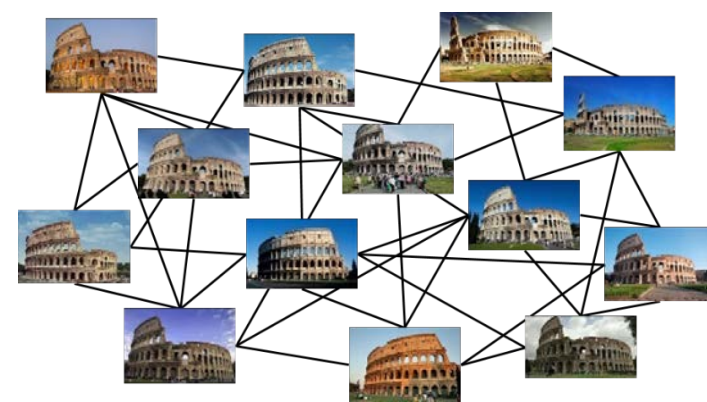
Structure from Motion

Unstructured Images



Assoc.

Scene Graph



SFM

Sparse Model

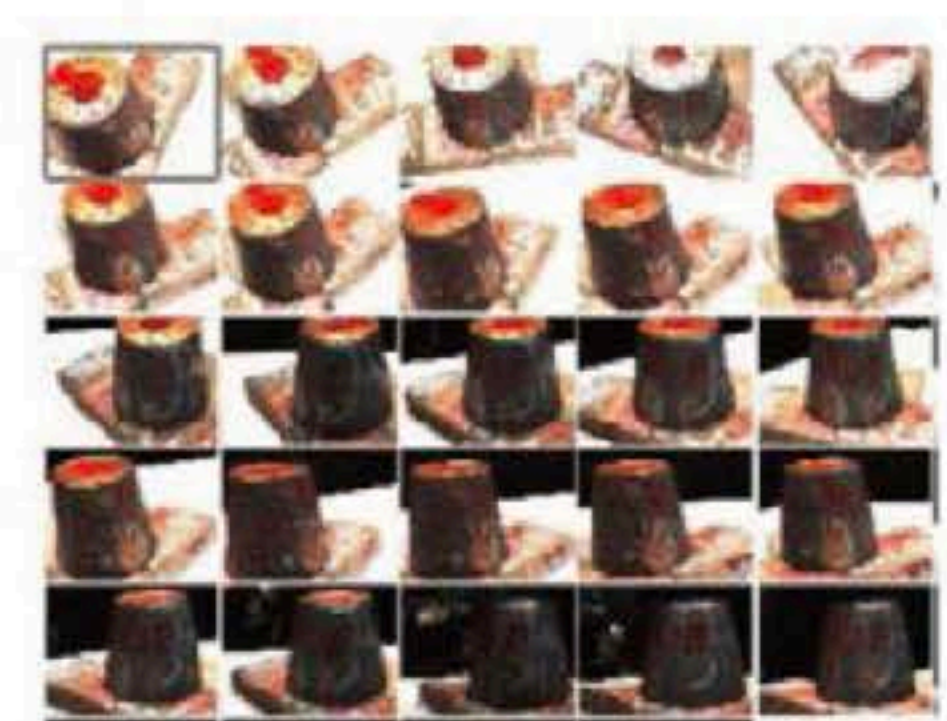


MVS

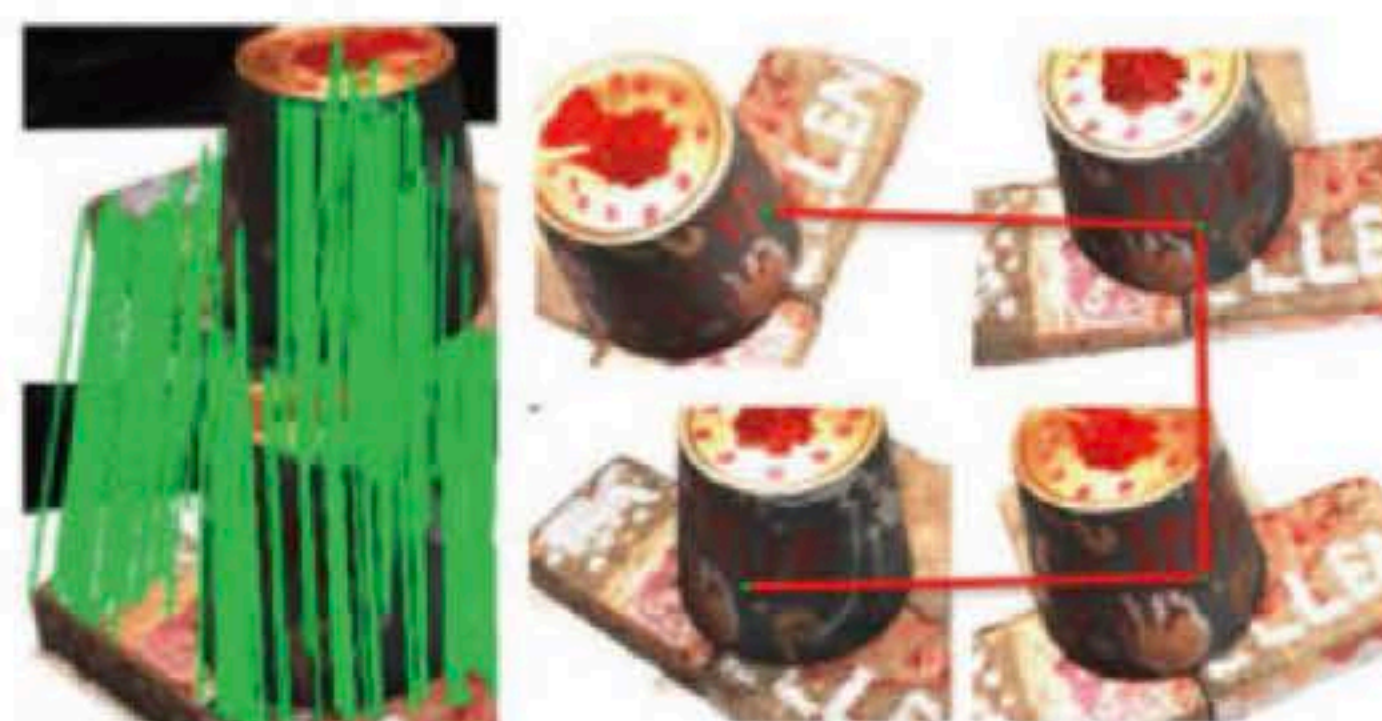
Dense Model



Learning-Based MVS



Input images



Find matched points
across images



Structure from
Motion (SFM)



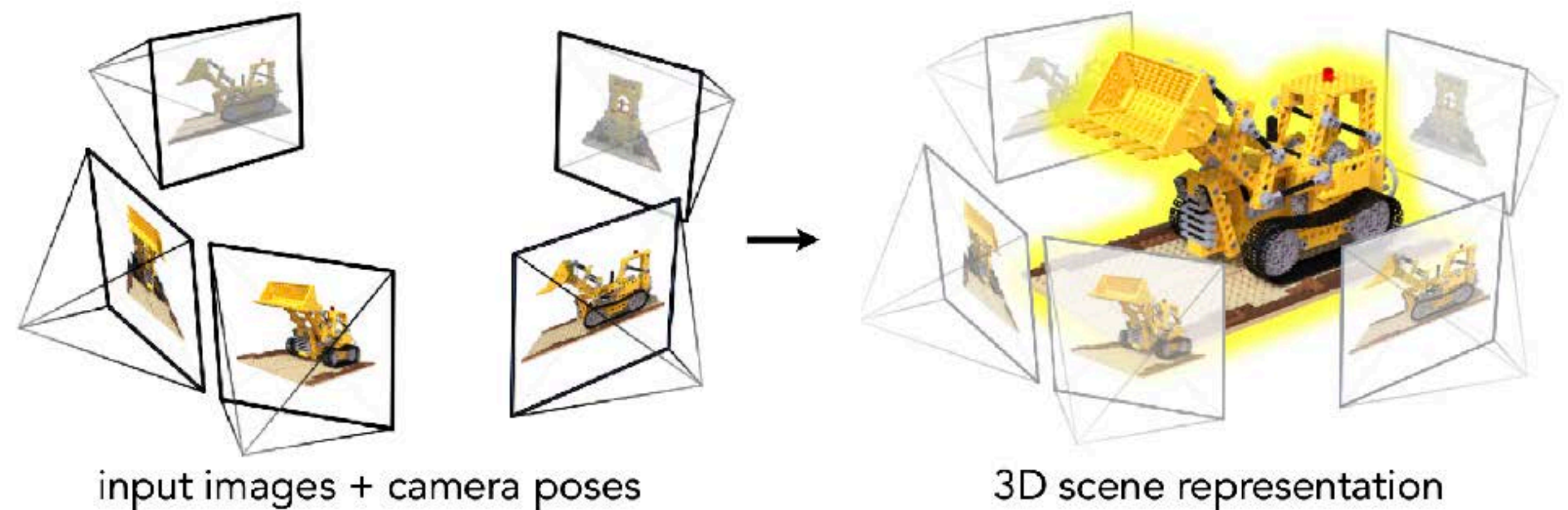
dense reconstruction

Learning-Based MVS

- Deep volumetric stereo can lead to more robust matching and more complete reconstruction
- But volume-based methods are NOT computationally efficient, since the 3D target scene is sparse
- Adaptive sampling can improve computation efficiency and reconstruction quality
- Normal prediction is easier than depth, and can help improve depth accuracy and smoothness

NeRF

- Neural Radiance Field (NeRF)
 - Implicit Functions: an Illustration with 3D Surface Representation
 - Volume Rendering with Ray Marching
 - Learning NeRF
- NeRF Extensions



Single Image to 3D

- Synthesis-for-learning pipeline leverages easy-to-obtain synthetic data for challenging 3D visual understanding tasks
- Single image to 3D point cloud is possible with properly defined set metric (EMD and CD)
- Natural ambiguity in single image to 3D
- Single image to mesh can be achieved through template deformation
- Mesh reconstruction requires more regularizations

3D Generative Models



Surface Reconstruction

	Explicit meshing (e.g., ball-pivoting)	Implicit meshing (e.g., RBF, MLS, Poission)
Sensitive to normals	No	Yes
Watertight manifold	No	Yes, in most cases
Complexity	Linear	• Large-scale equations to estimate implicit function • Marching cubes • Dense voxelization

3D Backbone Networks

- Voxel Networks
 - Projection
 - Sparse Convolution
- Point Networks
 - PointNet
 - PointNet++
 - Sampling Issues
- Graph Networks
 - Geodesic CNN
 - TextureNet

3D Detection and Segmentation

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets — Leveraging image proposal for 3D detection
 - ✦ PointPillars — Projecting 3D into 2D with a strong assumption
 - ✦ VoteNet — Clustering based proposal
- 3D Instance Segmentation
 - Top-Down Approaches — Proposal generation + point association
 - Bottom-Up Approaches — Careful feature learning for grouping purposes

6D Pose Estimation

- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches — rotation representation, loss, example: dense fusion
 - Indirect approaches — PVN3D, NOCS

Intrinsic Shape Understanding

- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction



Thank you for the efforts of this semester!